

NucNetPy

Release 1.0.0

M. H. Talafha and N. M. Ershaidat

Oct 06, 2026

1	Installation	3
1.1	Requirements	3
1.2	From a checkout	3
1.3	Optional extras	3
1.4	Verifying the installation	3
1.5	Building this manual.	4
2	Concepts	5
2.1	Species.	5
2.2	Reaction	5
2.3	ReactionNetwork.	6
2.4	Zone	6
2.5	Network.	6
2.6	Units	6
3	Common workflows	7
3.1	Reading a JINA/libnucnet database	7
3.2	Cutting a network down to size	7
3.3	Evolving one zone	8
3.4	Equilibrium.	8
3.5	Thermodynamic consistency	8
3.6	Screening	9
3.7	Entropy	9
3.8	Energy release.	10
3.9	Analysis	10
3.10	Command line	10
4	Pitfalls	11
4.1	Dropping the photon when you cut a network	11
4.2	Assuming reverse rates are consistent with the masses	11
4.3	Placeholder nuclear data in an equilibrium solve	11
4.4	Trusting <code>success</code> without checking it.	12
4.5	Forgetting the spin degeneracy	12
4.6	Tolerances	12
4.7	Interpolating a detailed-balance reverse rate	12
5	Verifying a calculation	13
5.1	What the package checks for you	13
5.2	What you should check on your own calculation	13
5.3	Golden files and what they do not prove.	14
5.4	Reproducing the published results	14

6	Python API	15
6.1	Core objects	15
6.2	Evolution.	23
6.3	Equilibrium	27
6.4	Detailed balance	32
6.5	Screening, weak rates, decays	35
6.6	Analysis	41
6.7	Input and output	45
6.8	Utilities.	47
7	Command line	57
7.1	summary.	57
7.2	print-output.	57
7.3	largest-x	58
7.4	zone-abundances	58
7.5	zone-properties	58
7.6	element-abundances	59
7.7	rates	59
7.8	flows	59
7.9	ydot	59
7.10	net-flows	60
7.11	charge-flows.	60
7.12	timescales	60
7.13	entropy-generation.	61
7.14	conservation.	61
7.15	remove-duplicates.	61
7.16	remove-invalid	61
7.17	export-zone-xml	62
7.18	reactions-latex	62
7.19	species-history	62
7.20	evolve-zone	62
7.21	energy-generation.	63
7.22	net-dot.	63
7.23	validate.	63
7.24	nse.	64
7.25	qse.	64
7.26	jina-summary.	64
7.27	jina-combine.	65
8	Conversion map from original NucNet Tools concepts to nucnetpy	67
8.1	What is implemented.	67
8.2	What still needs project-specific validation	68
9	Blog workflow coverage	69
9.1	Covered	69
9.2	Ported from the C++ source (r647)	71
9.3	Not ported (by design or still open).	72
10	Pure Python port status	73
10.1	What remains for strict numerical identity.	74
11	Exact numerical identity policy	75
11.1	Why identity cannot simply be assumed.	75
11.2	How to compare with a C++ build you run yourself	75
11.3	Development rule.	76

12 JINA XML workflow in nucnetpy	77
12.1 Files.	77
12.2 Python usage	77
12.3 Command-line usage.	77
12.4 Important numerical point.	77
13 Where to start	79
14 Citing	81
Python Module Index	83
Index	85

NucNetPy is a pure-Python nuclear reaction-network package. It reads JINA/libnucnet XML data, evaluates reaction rates, builds and integrates the abundance equations with stiff solvers, solves nuclear statistical and quasi-statistical equilibrium, and provides the analysis quantities used in the NucNet Tools workflows.

It is a clean reimplementation: it does not call or wrap the original C/C++ code, and NumPy and SciPy are its only mandatory dependencies.

Installation

1.1 Requirements

Python 3.9 or later, with NumPy 1.22+ and SciPy 1.9+. Nothing else is required to use the package.

1.2 From a checkout

```
git clone https://github.com/mtalafha90/NucNetPy.git
cd NucNetPy
python -m pip install -e .
pytest -q
```

The test suite runs in about a second and should report all tests passing before you rely on any result.

1.3 Optional extras

Extra	Adds	For
dev	pytest	running the test suite
hdf5	h5py	HDF5 persistence
plot	matplotlib, networkx	plotting and graph export
notebook	jupyter, pandas, matplotlib	the tutorial notebooks
docs	sphinx, furo, myst-parser	building this manual

```
python -m pip install -e ".[dev,hdf5,plot,notebook]"
```

pandas is used only by the notebooks to display tables; the package itself never imports it.

1.4 Verifying the installation

```
import nucnetpy as nn
print(nn.__version__)
```

The command-line interface is installed alongside:

```
nucnetpy --help
```

1.5 Building this manual

```
python -m pip install -e ".[docs]"  
sphinx-build -b html docs/manual docs/manual/_build/html
```

For a single self-contained file, suitable for a submission archive:

```
sphinx-build -b singlehtml docs/manual docs/manual/_build/singlehtml
```

Concepts

Five objects carry everything.

2.1 Species

A nuclide: name, charge z , mass number a , mass excess, ground-state spin, and an optional partition-function table.

```
import nucnetpy as nn

ni56 = nn.Species("ni56", z=28, a=56, mass_excess=-53.907, spin=0.0)
```

Mass excess and spin are not decoration. Equilibrium, detailed balance and energy release are all computed from mass excesses, and the statistical weight in an equilibrium solve is $(2J+1) * G(T)$, so a placeholder value produces a placeholder answer. A species whose spin is unknown is given a weight of one rather than a guess.

`Species.parse` builds one from a name alone, normalising aliases ($p \rightarrow h1$, $\alpha \rightarrow he4$), but it cannot invent nuclear data.

2.1.1 Participants that are not nuclides

`gamma`, `electron`, `positron` and the neutrinos appear in reaction records so that charge and lepton number balance, but they carry no baryon number and are never part of the abundance vector. `nucnetpy.species.is_massless` identifies them. See [Pitfalls](#).

2.2 Reaction

Reactants, products, a rate, a Q -value, and metadata. The rate may be a ReacLib seven-coefficient fit, a tabulated rate, a constant, or a callable; a reaction may carry more than one, and they add.

```
r = nn.Reaction.from_names(["c12", "he4"], ["o16", "gamma"],
                           rate_fits=[nn.RateFit([10, 0, 0, 0, 0, 0, 0])],
                           q_value=7.162)
```

The flow of a reaction is

(2.1)

where the products run over the reactants that carry baryon number, and i is that reactant order. `Reaction.nuclear_reactants` is the list actually used.

2.3 ReactionNetwork

A collection of reactions, with duplicate removal, filtering, and the construction of rates, flows, and .

2.4 Zone

One composition together with its thermodynamic properties. Abundances are molar: Y is moles per gram, and the mass fraction is $X = A \cdot Y$. A normalised composition has $\sum(X) = 1$.

```
zone = nn.Zone(abundances={"si28": 1.0 / 28.0},
               properties={"t9": "5.0", "rho": "1e8"})
```

2.5 Network

Species, reactions and zones in one container, plus `validate()`, which reports species named by a reaction but absent from the species map, reactions that do not balance, and species carrying no nuclear data.

2.6 Units

Quantity	Unit
Temperature t_9	K
Density ρ	g cm^{-3}
Abundance Y	mol g^{-1}
Mass excess	MeV
Q-value	MeV
Rate	s^{-1} scaled by the density powers of the flow
Energy generation	$\text{erg g}^{-1} \text{s}^{-1}$
Time	s

Common workflows

3.1 Reading a JINA/libnucnet database

```
import nucnetpy as nn

net = nn.read_jina_xml("nuclides.xml", "reaction_data.xml", zones_xml="zone.xml")
report = net.validate()
print(len(net.species), "species in memory,",
      len(net.reactions.reactions), "reactions")
print("without nuclear data:", len(report["species_without_nuclear_data"]))
```

Count carefully here. `len(net.species)` is the size of the in-memory network after parsing, not the number of entries in the nuclide file. Anything named by a reaction record with no entry in that file gets a synthesised placeholder, so it is present in memory whether or not the file described it.

On the JINA database used throughout this manual the nuclide file holds 7852 nuclides, while `len(net.species)` is 8131: those 7852 plus 279 placeholders. `validate()` reports no missing species precisely because the placeholders already exist, so `species_without_nuclear_data` is the field to read, not `missing_species`.

The two containers also differ. `net.species` holds nuclides only, while `net.species_names()` returns 8136 — the same 8131 plus the massless reaction participants `gamma`, `electron`, `positron` and the two neutrinos, which balance a reaction record but never enter the abundance vector.

Reading the database takes 20 to 30 s and holds it in about 0.9 GB, for a process peak near 1 GB. That is the practical ceiling for a pure-Python implementation, which is why the next step matters.

3.2 Cutting a network down to size

A one-zone calculation rarely needs the whole chart of nuclides.

```
from nucnetpy.network_limiter import select_species, limit_network

limit_network(net, select_species(net, zmax=20, amax=44))
```

`limit_network` keeps every reaction whose nuclides all survive the cut. Photons and leptons never decide the outcome, so the photodisintegrations stay; listing `"gamma"` is harmless. See [Pitfalls](#) for why this matters.

3.3 Evolving one zone

```
result = nn.evolve_zone(
    net, net.zone(0), nn.time_grid(0.0, 10.0, 120),
    thermo=nn.constant_thermo(t9=5.0, rho=1.0e8),
    method="bdf", rtol=1e-8, atol=1e-14,
)
assert result.success, result.message
```

thermo is any callable (t, abundances) -> (T9, rho), so a prescribed temperature and density history drops straight in; `nucnetpy.hydro.Trajectory` wraps a tabulated one with interpolation.

Supported methods are `bdf`, `radau`, `lsoda`, `rk45` and `dop853`, plus fixed-step `rk4`, `euler` and `implicit_euler` fallbacks. Reaction networks are stiff, so the first three are the practical choices.

3.3.1 Choosing the Jacobian

The flows are monomials in the abundances, so the Jacobian is available in closed form and costs one sweep over the reaction list, against $N+1$ right-hand-side evaluations for finite differences.

```
nn.evolve_zone(..., jac_mode="analytic")    # default; >100x faster at 100 species
nn.evolve_zone(..., jac_mode="numerical")  # finite differences
nn.evolve_zone(..., jac_mode="sparsity")    # SciPy estimates it from the pattern
```

SciPy applies a sparsity pattern only when no Jacobian callable is supplied, so the pattern and an explicit Jacobian are alternatives rather than complements.

3.4 Equilibrium

```
nse = nn.solve_nse(net, t9=5.0, rho=1.0e8, ye=0.5)
print(nse.success, nse.xsum, nse.computed_ye)
```

The solve shares no machinery with the time integration, which is what makes it a genuine check on a network calculation — provided the network is running on the rate library's own reverse rates. See the caveat under *Thermodynamic consistency* below. Species carrying no nuclear data are excluded automatically; pass `require_nuclear_data=False` to override that, and read [Pitfalls](#) before doing so.

Constrained cluster equilibria (the `libnuceq` workflow) use `solve_qse` with `QSECluster` constraints. Plasma corrections are available through `nucnetpy.coulomb.nse_correction`.

3.5 Thermodynamic consistency

Rate libraries fit the forward and reverse directions of a reaction separately, so a network built from them relaxes to a state that is not the NSE of the same nuclear data.

```
consistent = nn.consistent_reverse_network(net)
```

This pairs each reaction with its inverse, keeps the exothermic member, and regenerates the other from detailed balance. On the JINA alpha chain it improves agreement between the integrated composition and a separately solved NSE from a median of 4.5 per cent to 7 parts in 10^8 , once the equilibrium is solved over the same species set the network can populate.

Read that second figure carefully. The regenerated reverse rates come from the same equilibrium prefactor `solve_nse` uses, so the two calculations now share their nuclear data: an error in the masses, the $(2J+1)$ factor or the partition-function convention would move both together and cancel. That

agreement measures how consistently the integrator, the stoichiometry and the equilibrium solver treat one shared formulation, not whether that formulation is right.

It is an option rather than a default, because it replaces measured reverse rates with values derived from mass differences. Pass `tabulate=True` if the network must remain writable to XML, at some cost in accuracy.

3.6 Screening

```
from nucnetpy import SkyNetScreening

nn.evolve_zone(..., screening=SkyNetScreening(net.species))
```

Each nuclear charge is assigned a Coulomb chemical potential $\mu(Z)/kT$, and a reaction is enhanced by the difference between the separated reactants and the fused compound charge. Being a chemical potential, it applies consistently to forward and reverse rates, so screening does not undo detailed balance. A pairwise Salpeter factor is also available through `reaction_screening_factor`.

The electrons screen too, by an amount that depends on how degenerate they are, and `SkyNetScreening` works it out from the electron gas (`nucnetpy.electrons`):

- for non-degenerate electrons, as in hydrogen and helium burning in ordinary stars, it recovers the Salpeter weak-screening limit;
- for degenerate electrons, as in a white dwarf, they no longer screen;
- in hot, thin matter, positron pairs add to the screening.

The electron state is recomputed only when the temperature, density or Y_e changes, which costs about a millisecond. To fix the electrons' share yourself, pass a number as `pair_term`: `pair_term=0.0` keeps the ions only, and `pair_term=Y_e` treats the electrons as non-degenerate.

3.7 Entropy

```
s = nn.entropy_per_nucleon(zone.abundances, net, t9=5.0, rho=1.0e8)
s.ions, s.electrons, s.photons, s.coulomb, s.total
```

The entropy per nucleon, in units of Boltzmann's constant, comes in parts:

- **ions** — the Sackur-Tetrode entropy of each species, with its real mass, ground-state spin and partition function (the last adds the entropy of the excited states);
- **electrons and positrons** — from the electron gas, at any degeneracy;
- **photons** — black-body radiation;
- **Coulomb** — the plasma correction, off unless `coulomb=True`.

To go the other way, `nn.density_for_entropy(s, abundances, net, t9)` and `nn.t9_for_entropy(s, abundances, net, rho)` find the density or temperature that gives entropy `s` at fixed composition. The entropy changes steadily with both, so the answer is unique.

`nn.constant_entropy_thermo(s, density, net)` turns this into a `thermo` callable: you give the density as a function of time, and the temperature is set at each step so that the current composition has entropy `s`. This is the usual way to follow matter as it expands and cools. The heat from the reactions is not fed back, so use it once the burning has slowed, as in a freeze-out. Burning that reduces the number of particles (helium to carbon, say) makes the temperature rise at fixed entropy.

3.8 Energy release

```
from nucnetpy import nuclear_energy_generation_rate, nuclear_energy_release
```

The rate follows from the change in total mass excess, $\text{eps} = -N_A * C * \sum_i (dY_i/dt) * dM_i$, where $C = 1.602176634\text{e-}6 \text{ erg/MeV}$ converts the mass excesses from MeV. It does not use the reaction Q -values stored in the rate library, so it is internally consistent with the adopted mass-excess table, the same one `solve_nse` uses. It may differ from library Q -values when the two data sources use different nuclear masses. Neutrino losses are not subtracted.

3.9 Analysis

`nucnetpy.analysis` carries the diagnostics of the NucNet Tools workflows: largest mass fractions, element abundances, abundance moments, per-species timescales $Y/|dY/dt|$, per-reaction contributions to dY_e/dt , entropy generation, s -process neutron exposure, integrated reaction currents, and separation energies.

3.10 Command line

Every operation above has a command-line equivalent; see [Command line](#).

Pitfalls

Everything in this chapter produces a plausible-looking answer rather than an error message. Each item was found by running the package against a production database rather than a test network, and each is now covered by a regression test — but the underlying trap belongs to the problem, not to this implementation, so it is worth knowing whichever code you use.

4.1 Dropping the photon when you cut a network

A code that keeps a reaction only if *all* its participants survive a cut, and cuts by a list of nuclides, discards every photodisintegration, because the photon is not a nuclide. Nothing warns you. The network simply cannot reach equilibrium, and a silicon-burning calculation will not start at all, because it begins with .

`limit_network` avoids this: only nuclides decide whether a reaction survives, so photons and leptons are kept automatically and listing "gamma" is harmless. Development versions of NucNetPy before the 1.0.0 release fell into the trap too, and with `include_linked=True` they also treated the photon as a link between otherwise unrelated reactions:

```
limit_network(net, select_species(net, zmax=20)) # keeps
the photodisintegrations
```

4.2 Assuming reverse rates are consistent with the masses

A network relaxes to the equilibrium implied by the **ratios of the rates it is given**, not to the equilibrium of the nuclear data. Rate libraries fit the two directions separately, so the two need not agree.

Evaluated at the NSE composition, JINA's reverse fits leave the net flux of a reaction far from zero; rebuilt from detailed balance, the net flux vanishes to machine precision. On the alpha chain at the fits differ from detailed balance by a median factor of 1.10 and up to 1.72, which displaces equilibrium mass fractions by a median of 4.5 per cent.

Use `consistent_reverse_network` if you need the network and the equilibrium solver to agree, and understand that you are then trusting the mass table over the library's reverse fit.

4.3 Placeholder nuclear data in an equilibrium solve

A production reaction file names nuclides that its nuclide file does not describe — 279 of them in the database used for the manuscript. NucNetPy synthesises placeholders so the reactions remain usable, with a mass excess of zero.

A mass excess of zero is not a neutral default: it makes an unbound nuclide look as tightly bound as the most stable one. Left in an NSE solve, Li reached a mass fraction of 0.28 and displaced the iron

peak. They are therefore recorded in `network.validate()["species_without_nuclear_data"]` and excluded from equilibrium solves by default.

4.4 Trusting success without checking it

```
result = nn.evolve_zone(...)
assert result.success, result.message
```

`success` is false when the solver failed, and also when positivity projection had to invent enough baryon number to break conservation by more than the requested tolerance.

That second case is worth understanding. The right-hand side clips its input to non-negative values, so once a component goes negative the derivative stops depending on it and its Jacobian column vanishes; an implicit solver is then free to carry that component far negative while still passing its own convergence test. Clipping it back to zero at the end converts a large negative number into invented mass. In one long integration this returned with `True`, while SciPy reported that it had reached the end of the interval successfully.

Checking conservation costs one inner product per output step. Do it.

4.5 Forgetting the spin degeneracy

libnucnet partition-function tables are *normalised*: they tend to one as g for every nuclide, high-spin ones included. The statistical weight is $2J+1$, and the degeneracy has to be supplied separately. Omitting it makes every nuclide with $J > 0$ too rare by that factor — up to 90 per cent for V .

NucNetPy applies it automatically when a species carries a spin. If your species were built with `Species.parse`, they do not.

4.6 Tolerances

The default `atol` is an absolute floor on abundances. Set it far below the smallest abundance you care about, but not so far that the solver chases numerical noise in species that are effectively absent — that is a common cause of a stiff solve failing to start at all. Confirm the answer has stopped moving:

```
for rtol in [1e-4, 1e-6, 1e-8, 1e-10]:
    r = nn.evolve_zone(..., rtol=rtol, atol=rtol * 1e-8)
```

4.7 Interpolating a detailed-balance reverse rate

The reverse rate carries $\exp(-E/kT)$, which is far too steep to interpolate. Tabulating it on 200 logarithmically spaced points costs three orders of magnitude of accuracy against evaluating it exactly. `consistent_reverse_network` attaches a callable by default for this reason; `tabulate=True` is for when the network must be serialisable.

Verifying a calculation

Two different questions are worth keeping apart.

Verification asks whether the code does what it was designed to do. *Validation* asks whether that design describes nature, and needs a reference outside the code.

5.1 What the package checks for you

The test suite covers XML round-tripping as a fixed point, ReacLib rates on a grid, screening factors, weak-rate interpolation, right-hand-side values and their baryon residual, full trajectories, the analytic Jacobian against finite differences, the treatment of photons and leptons, solver dispatch, the spin degeneracy, exclusion of species without nuclear data, and detection of a positivity projection that manufactures mass.

```
pytest -q
```

5.2 What you should check on your own calculation

Conservation. For a reaction set that conserves baryon number, the instantaneous residual (5.1)

should stay at round-off, and δ should stay at its initial value. Track them along the whole trajectory, not only at the start.

These are two different quantities and are worth keeping apart when you quote them. δ is a **rate**, in : how fast the right-hand side fails to conserve baryon number at one instant. The departure of δ from unity is a **dimensionless** normalisation error of the composition itself. A third quantity — the baryon number a positivity projection had to manufacture, reported when a solve fails — is a dimensionless fraction of the initial amount, different again. On a well-behaved problem all three sit near round-off, which makes them easy to conflate.

Solver agreement. Repeat the calculation with `bdf`, `radau` and `lsoda`. They should agree far more closely than any physical effect you care about.

Tolerance convergence. Tighten `rtol` and confirm the endpoint stops moving.

Equilibrium. Where the physics allows it, run the calculation to its stationary state and compare with `solve_nse`. This is the strongest check available, but what it tests depends on where the reverse rates came from, and the two cases are worth keeping apart.

With the library's own reverse fits the two sides share no numerical input, so a disagreement is informative — though it will usually measure the rate library rather than the code, which is why [Pitfalls](#) on reverse rates is worth reading first.

With reverse rates rebuilt by `consistent_reverse_network` the two sides are no longer indepen-

dent: the reverse rates are derived from the same equilibrium prefactor `solve_nse` uses, so the masses, the $(2J+1)$ factor and the partition-function convention are common to both and cancel from the comparison. That agreement is a strong end-to-end test of the integrator, the stoichiometry and the equilibrium solver's root finding, and says nothing about whether the shared formulation is right. For that you need a reference outside this package.

5.3 Golden files and what they do not prove

`tests/golden/` holds frozen outputs, and `nucnetpy.validation.regression_summary` compares against them. These detect accidental changes in behaviour. They are snapshots of this implementation, so they are not independent evidence that an answer is right, and they must not be described as agreement with the original C++ code.

`validation/generate_golden.py` writes the reference records in the format the suite consumes, so outputs from a NucNet Tools build can be substituted for the self-generated ones without changing any test code.

5.4 Reproducing the published results

```
python validation/demonstration.py \  
    --nuclides nuclides.xml --reactions reaction_data.xml --zone zone.xml \  
    --outdir results/  
  
python validation/benchmark_performance.py \  
    --nuclides nuclides.xml --reactions reaction_data.xml \  
    --json results/benchmark.json --latex results/table_benchmark.tex
```

Both record the interpreter, NumPy, SciPy and platform versions alongside their output.

Everything below is exported at the top level of `nucnetpy` unless noted. Pages are generated from the docstrings in the source.

6.1 Core objects

Nuclide/species utilities.

This module provides the most frequently needed pieces of libnucnet species handling in pure Python: name parsing, Z/A/N bookkeeping, mass fractions, charge fractions, and common element symbols.

```
nucnetpy.species.MASSLESS_SPECIES = frozenset({'anti-neutrino_e', 'antineutrinoe', 'electron', 'gamma', 'neutrino_e', 'neutrinoe', 'positron'})
```

Reaction participants that carry no baryon number. Photons and leptons appear explicitly in JINA/libnucnet reaction records so that charge and lepton number can be checked, but they are not part of the abundance vector: the photon bath and the electron sea are properties of the thermodynamic state, and their densities are already folded into the tabulated rate. They must therefore be excluded from the abundance product, the reactant order, and the identical-particle statistical factor of a reaction flow.

```
nucnetpy.species.is_massless ( name )
```

Return True for photons and leptons, which carry no baryon number.

Parameters `name` (*str*)

Return type bool

```
class nucnetpy.species.Species ( name, z, a, mass_excess=0.0, spin=None, source=None, partition=<factory> )
```

Bases: object

A nuclear species.

Parameters are intentionally similar to libnucnet's Nuc objects. *mass_excess* is in MeV when available. *partition* may store temperature dependent partition-function data.

Parameters

- **name** (*str*)
- **z** (*int*)
- **a** (*int*)
- **mass_excess** (*float*)
- **spin** (*float* | *None*)
- **source** (*str* | *None*)
- **partition** (*Dict*[*float*, *float*])

```
name: str

z: int

a: int

mass_excess: float = 0.0

spin: float | None = None

source: str | None = None

partition: Dict[float, float]

property n: int

property element: str

property latex: str

classmethod parse ( value, **kwargs )
    Parameters value(str)
    Return type Species

nucnetpy.species.normalize_species_name ( value )
    Parameters value(str)
    Return type str

nucnetpy.species.species_from_za ( z, a, **kwargs )
    Parameters
        • z(int)
        • a(int)

    Return type Species

nucnetpy.species.mass_fraction ( y, a )
    Parameters
        • y(float)
        • a(int)

    Return type float

nucnetpy.species.abundance_from_mass_fraction ( x, a )
    Parameters
        • x(float)
        • a(int)

    Return type float
```

Core containers: Network, Zone, and multi-zone collections.

```
class nucnetpy.core.Zone ( label: 'Tuple[str, str, str]'=('0', '0', '0'),
    abundances: 'Dict[str, float]'=<factory>, properties: 'Dict[str,
    str]'=<factory>, optional_properties: 'Dict[str, str]'=<factory> )
    Bases: object

    Parameters
        • label(Tuple[str, str, str])
        • abundances(Dict[str, float])
```

```

    • properties (Dict[str, str])
    • optional_properties (Dict[str, str])

label: Tuple[str, str, str] = ('0', '0', '0')
abundances: Dict[str, float]
properties: Dict[str, str]
optional_properties: Dict[str, str]
set_abundance ( species, y )
    Parameters    • species (str)
                  • y (float)

    Return type None

get_abundance ( species, default=0.0 )
    Parameters    • species (str)
                  • default (float)

    Return type float

mass_fractions ( species_map=None )
    Parameters species_map (Mapping[str, Species] | None)
    Return type Dict[str, float]

ye ( species_map=None )
    Parameters species_map (Mapping[str, Species] | None)
    Return type float

ysum ( )
    Return type float

xsum ( species_map=None )
    Parameters species_map (Mapping[str, Species] | None)
    Return type float

temperature9 ( default=1.0 )
    Parameters default (float)
    Return type float

density ( default=1.0 )
    Parameters default (float)
    Return type float

class nucnetpy.core.Network ( species: 'Dict[str, Species]'=<factory>,
    reactions: 'ReactionNetwork' = <factory>, zones: 'List[Zone]' = <factory>,
    metadata: 'Dict[str, str]'=<factory> )
    Bases: object
    Parameters    • species (Dict[str, Species])

```

- **reactions** (*ReactionNetwork*)
- **zones** (*List[Zone]*)
- **metadata** (*Dict[str, str]*)

species: *Dict[str, Species]*

reactions: *ReactionNetwork*

zones: *List[Zone]*

metadata: *Dict[str, str]*

add_species (*species*)

Parameters **species** (*Species*)

Return type *None*

ensure_species (*name*)

Parameters **name** (*str*)

Return type *Species*

add_zone (*zone*)

Parameters **zone** (*Zone*)

Return type *None*

species_names ()

Return type *List[str]*

zone (*index=0*)

Parameters **index** (*int*)

Return type *Zone*

validate ()

Return type *Dict[str, object]*

species_without_nuclear_data ()

Return species that were synthesised to satisfy a reaction record.

These are referenced by the reaction file but absent from the nuclide file, so they carry a placeholder mass excess of zero rather than measured or evaluated data. They are safe for structural operations such as counting or plotting the network, but must not enter an equilibrium calculation, where a mass excess sets the binding energy.

Return type *List[str]*

abundance_matrix (*species=None*)

Parameters **species** (*Sequence[str] | None*)

Return type *Tuple[List[str], ndarray]*

Reaction and rate machinery for a pure-Python NucNet Tools replacement.

The implementation supports common ReacLib 7-coefficient rates, tabulated rates, manual constants, duplicate handling, conservation checks, and stoichiometric flux evaluation suitable for one-zone evolution and analysis commands.


```
class nucnetpy.reactions.ReactionParticipant ( species: 'str', count:
'int' = 1 )
```

```
    Bases: object
```

```
    Parameters    • species (str)
                  • count (int)
```

```
    species: str
```

```
    count: int = 1
```

```
class nucnetpy.reactions.RateFit ( coefficients, label='', chapter=None )
```

```
    Bases: object
```

```
    A standard seven-parameter ReacLib rate fit.
```

```
    lambda =  $\exp(a_0 + a_1/T_9 + a_2/T_9^{1/3} + a_3*T_9^{1/3} + a_4*T_9$ 
```

```
        •  $a_5*T_9^{5/3} + a_6*\ln(T_9)$ )
```

A reaction may carry several such fits, and `Reaction.bare_rate` sums them. The result is a generic rate coefficient, not $N_A\langle\sigma v\rangle$: the same form serves one-body decays and photo-disintegrations, two-body captures and three-body reactions, so its units depend on the number of nuclear reactants. The density factor in `Reaction.flux` converts it to an abundance flow.

```
    Parameters    • coefficients (Sequence[float])
                  • label (str)
                  • chapter (int | None)
```

```
    coefficients: Sequence[float]
```

```
    label: str = ''
```

```
    chapter: int | None = None
```

```
    rate ( t9 )
```

```
        Parameters t9 (float)
```

```
        Return type float
```

```
class nucnetpy.reactions.TabularRate ( t9: 'Sequence[float]', rates:
'Sequence[float]', log_interp: 'bool' = True )
```

```
    Bases: object
```

```
    Parameters    • t9 (Sequence[float])
                  • rates (Sequence[float])
                  • log_interp (bool)
```

```
    t9: Sequence[float]
```

```
    rates: Sequence[float]
```

```
    log_interp: bool = True
```

```
    rate ( t9 )
```

```
        Parameters t9 (float)
```

```
        Return type float
```

```
class nucnetpy.reactions.Reaction ( reactants: 'List[ReactionParticipan-
t]', products: 'List[ReactionParticipant]', rate_fits: 'List[RateFit]' =
<factory>, tabular_rate: 'Optional[TabularRate]' = None, q_value: 'float' =
0.0, source: 'str' = '', label: 'str' = '', metadata: 'Dict[str, str]' =
<factory>, constant_rate: 'Optional[float]' = None, rate_function: 'Option-
al[Callable[[float], float]]' = None )
```

Bases: object

Parameters

- **reactants** (*List[ReactionParticipant]*)
- **products** (*List[ReactionParticipant]*)
- **rate_fits** (*List[RateFit]*)
- **tabular_rate** (*TabularRate | None*)
- **q_value** (*float*)
- **source** (*str*)
- **label** (*str*)
- **metadata** (*Dict[str, str]*)
- **constant_rate** (*float | None*)
- **rate_function** (*Callable[[float], float] | None*)

reactants: *List[ReactionParticipant]*

products: *List[ReactionParticipant]*

rate_fits: *List[RateFit]*

tabular_rate: *TabularRate | None = None*

q_value: *float = 0.0*

source: *str = ''*

label: *str = ''*

metadata: *Dict[str, str]*

constant_rate: *float | None = None*

rate_function: *Callable[[float], float] | None = None*

Optional $f(t_9) \rightarrow$ rate evaluated exactly at the requested temperature. It is used for rates that are computed rather than fitted or tabulated, such as the detailed-balance reverse rates built by `nucnetpy.consistent_reverse_network()`, whose steep exponential temperature dependence is poorly captured by interpolation. A callable cannot be written to XML, so a network containing one does not round-trip; tabulate the rate instead when persistence is required.

classmethod from_names (*reactants, products, **kwargs*)

Parameters

- **reactants** (*Sequence[str]*)
- **products** (*Sequence[str]*)

Return type *Reaction*

property key: *Tuple[Tuple[Tuple[str, int], ...], Tuple[Tuple[str, int], ...]]*

property string: `str`

property latex: `str`

property nuclear_reactants: `List[ReactionParticipant]`

Reactants that carry baryon number.

Photons and leptons are dropped: a photodisintegration such as $\text{si28} + \gamma \rightarrow \text{mg24} + \text{he4}$ is a one-body process whose rate already contains the photon density, so including γ in the abundance product would multiply every such flow by $Y(\gamma) = 0$ and would raise the density power by one.

property reactant_order: `int`

bare_rate (*t9*, *rho*=1.0, *ye*=None)

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float* | *None*)

Return type `float`

rate (*t9*, *rho*=1.0, *ye*=None, *screening*=None)

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float* | *None*)
- **screening** (*Callable*[[*Reaction*, *float*, *float*, *float* | *None*], *float*] | *None*)

Return type `float`

statistical_factor ()

Return type `int`

flux (*abundances*, *t9*, *rho*=1.0, *screening*=None, *ye*=None)

Parameters

- **abundances** (*Mapping*[*str*, *float*])
- **t9** (*float*)
- **rho** (*float*)
- **screening** (*Callable*[[*Reaction*, *float*, *float*, *float* | *None*], *float*] | *None*)
- **ye** (*float* | *None*)

Return type `float`

stoichiometry ()

Return type *Dict*[*str*, *int*]

conserves_a_z (*species_map*)

Parameters **species_map** (*Mapping*[*str*, *Species*])

Return type *Tuple*[*bool*, *int*, *int*]

```
class nucnetpy.reactions.ReactionNetwork ( reactions: 'List[Reaction]' =
<factory> )
    Bases: object

    Parameters reactions (List[Reaction])

    reactions: List[Reaction]

    add ( reaction )
        Parameters reaction (Reaction)
        Return type None

    extend ( reactions )
        Parameters reactions (Iterable[Reaction])
        Return type None

    remove_duplicates ( )
        Return type ReactionNetwork

    invalid_reactions ( species_map )
        Parameters species_map (Mapping[str, Species])
        Return type List[Reaction]

    filter_valid ( species_map )
        Parameters species_map (Mapping[str, Species])
        Return type ReactionNetwork

    rates ( t9, rho=1.0 )
        Parameters
            • t9 (float)
            • rho (float)

        Return type Dict[str, float]

    flows ( abundances, t9, rho=1.0, screening=None, ye=None )
        Parameters
            • abundances (Mapping[str, float])
            • t9 (float)
            • rho (float)
            • screening (Callable[[Reaction, float, float, float] |
                None], float] | None)
            • ye (float | None)

        Return type Dict[str, float]

    ydot ( abundances, t9, rho=1.0, screening=None, ye=None )
        Return dY/dt for every nuclide the reactions touch.
        Photons and leptons are left out: they are not part of the abundance vector, so a dY/dt for
        them would be a meaningless number that downstream diagnostics would report as if it
        were a species.

        Parameters
            • abundances (Mapping[str, float])
            • t9 (float)
            • rho (float)
```

- **screening** (*Callable*[[*Reaction*, *float*, *float*, *float* | *None*], *float*] | *None*)
- **ye** (*float* | *None*)

Return type *Dict*[*str*, *float*]

species_names ()

Return type *List*[*str*]

class `nucnetpy.reactions.DetailedBalanceOptions` (*t9*, *rho*, *ye*, *equilibrium_ratio*=1.0)

Bases: *object*

Options for estimating reverse rates from forward rates.

Exact NucNet matching requires identical masses, partition functions, and Coulomb corrections. This helper provides a practical replacement hook.

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **equilibrium_ratio** (*float*)

t9: *float*

rho: *float*

ye: *float*

equilibrium_ratio: *float* = 1.0

`nucnetpy.reactions.reverse_rate_from_detailed_balance` (*forward_rate*, *equilibrium_ratio*)

Return reverse rate using $\lambda_{\text{rev}} = \lambda_{\text{fwd}} / K_{\text{eq}}$.

Parameters

- **forward_rate** (*float*)
- **equilibrium_ratio** (*float*)

Return type *float*

`nucnetpy.reactions.read_reacLib_text` (*text*)

Read a small ReacLib-like text block.

This parser accepts common two-line ReacLib records and a compact one-line form: *reactants* -> *products* ; *a0* ... *a6*. It is intentionally permissive for migration from C++ examples and hand-written test networks.

Parameters **text** (*str*)

Return type *ReactionNetwork*

6.2 Evolution

One-zone and multi-zone evolution tools.

This layer implements the standard stoichiometric ODE interface and provides fixed-step, SciPy BDF /Radau/LSODA, optional sparse Jacobians, positivity projection, screening hooks, and weak-rate source terms.

```
nucnetpy.solver.METHODS = ('bdf', 'radau', 'lsoda', 'rk45', 'dop853',  
'rk4', 'euler', 'implicit_euler', 'backward_euler')
```

Every method name accepted by `evolve_zone()` (case-insensitive).

```
nucnetpy.solver.constant_thermo ( t9=1.0, rho=1.0 )
```

Parameters

- `t9` (*float*)
- `rho` (*float*)

Return type *Callable*[[*float*, *Mapping*[*str*, *float*]], *Tuple*[*float*, *float*]]

```
nucnetpy.solver.zone_thermo ( zone )
```

Parameters `zone` (*Zone*)

Return type *Callable*[[*float*, *Mapping*[*str*, *float*]], *Tuple*[*float*, *float*]]

```
nucnetpy.solver.time_grid ( t0, t1, n, log=False )
```

Parameters

- `t0` (*float*)
- `t1` (*float*)
- `n` (*int*)
- `log` (*bool*)

Return type *ndarray*

```
class nucnetpy.solver.EvolutionResult ( time: 'np.ndarray', species:  
'List[str]', y: 'np.ndarray', success: 'bool' = True, message: 'str' = '',  
nfev: 'int' = 0, njev: 'int' = 0 )
```

Bases: *object*

Parameters

- `time` (*ndarray*)
- `species` (*List*[*str*])
- `y` (*ndarray*)
- `success` (*bool*)
- `message` (*str*)
- `nfev` (*int*)
- `njev` (*int*)

`time`: *ndarray*

`species`: *List*[*str*]

`y`: *ndarray*

`success`: *bool* = *True*

`message`: *str* = ''

`nfev`: *int* = 0

`njev`: *int* = 0

`property final_abundances`: *Dict*[*str*, *float*]

zone ()

Return type *Zone*

mass_fraction_history (*species_map=None*)

Return type *ndarray*

`nucnetpy.solver.rhs` (*network, species, thermo, screening=None, weak_rates=None*)

Parameters

- **network** (*Network*)
- **species** (*Sequence[str]*)
- **thermo** (*Callable[[float, Mapping[str, float]], Tuple[float, float]]*)
- **screening** (*Callable[[object, float, float, float | None], float] | None*)
- **weak_rates** (*Sequence[object] | None*)

`nucnetpy.solver.jacobian` (*network, species, thermo, screening=None, weak_rates=None, eps=1e-08, sparse=False*)

Parameters

- **network** (*Network*)
- **species** (*Sequence[str]*)
- **thermo** (*Callable[[float, Mapping[str, float]], Tuple[float, float]]*)
- **screening** (*Callable[[object, float, float, float | None], float] | None*)
- **weak_rates** (*Sequence[object] | None*)
- **eps** (*float*)
- **sparse** (*bool*)

`nucnetpy.solver.analytic_jacobian` (*network, species, thermo, screening=None, weak_rates=None, sparse=True*)

Return an analytic Jacobian of the stoichiometric right-hand side.

Differentiating the flow of Eq. (flow),

$$F_r = (\lambda_r \rho^{(n_r-1)} / \prod_i m_{ir}) \prod_i Y_i^{m_{ir}},$$

with respect to a reactant abundance gives

$$\frac{dF_r}{dY_j} = m_{jr} Y_j^{m_{jr}-1} (\lambda_r \rho^{(n_r-1)} / \prod_i m_{ir}) \prod_{i \neq j} Y_i^{m_{ir}},$$

so that $J_{ij} = \sum_r \nu_{ir} \frac{dF_r}{dY_j}$. The partial product is formed explicitly rather than as $F_r m_{jr} / Y_j$ so that the derivative stays correct when Y_j vanishes, which is the usual state of most species at the start of a calculation.

The whole matrix costs one sweep over the reaction list, whereas the finite-difference Jacobian of `jacobian()` costs $N+1$ evaluations of the full right-hand side. The saving therefore grows linearly with the number of evolved species.

λ_r is treated as independent of composition. With a screening function or weak-rate tables that depend on Y the result is an approximate Jacobian: it changes the Newton convergence rate of an implicit solver but not the solution it converges to.

Parameters

- **network** (*Network*)
- **species** (*Sequence[str]*)

- **thermo** (*Callable*[[float, Mapping[str, float]], Tuple[float, float]])
- **screening** (*Callable*[[object, float, float, float | None], float] | None)
- **weak_rates** (*Sequence*[object] | None)
- **sparse** (*bool*)

nucnetpy.solver.jacobian_sparsity (network, species)

- Parameters
- **network** (*Network*)
 - **species** (*Sequence*[str])

nucnetpy.solver.evolve_zone (network, zone, times, thermo=None, method='bdf', species=None, screening=None, weak_rates=None, rtol=1e-06, atol=1e-30, use_jacobian=True, project_positive=True, jac_mode='analytic')

Evolve one zone over times.

`jac_mode` selects how the Jacobian is supplied to the implicit SciPy solvers. "analytic" (the default) uses `analytic_jacobian()`, which differentiates the stoichiometric flows in closed form and costs one sweep over the reaction list. "numerical" uses the finite-difference `jacobian()`, costing N+1 right-hand-side evaluations. "sparsity" supplies no Jacobian and instead passes the stoichiometric sparsity pattern so that SciPy estimates the matrix by grouped finite differences.

Note that SciPy applies `jac_sparsity` only when no Jacobian callable is given, so the pattern and an explicit Jacobian are alternatives rather than complements.

`method` is one of `METHODS`, in any case. An unrecognised name raises `ValueError` rather than falling back to another integrator.

- Parameters
- **network** (*Network*)
 - **zone** (*Zone*)
 - **times** (*Sequence*[float])
 - **thermo** (*Callable*[[float, Mapping[str, float]], Tuple[float, float]] | None)
 - **method** (*str*)
 - **species** (*Sequence*[str] | None)
 - **screening** (*Callable*[[object, float, float, float | None], float] | None)
 - **weak_rates** (*Sequence*[object] | None)
 - **rtol** (*float*)
 - **atol** (*float*)
 - **use_jacobian** (*bool*)
 - **project_positive** (*bool*)
 - **jac_mode** (*str*)

Return type *EvolutionResult*

nucnetpy.solver.evolve_network_zones (network, times, thermo=None, method='bdf', **kwargs)

- Parameters
- **network** (*Network*)
 - **times** (*Sequence*[float])

- **thermo** (*Callable*[[*float*, *Mapping*[*str*, *float*]], *Tuple*[*float*, *float*]] | *None*)
- **method** (*str*)

Return type *List*[*EvolutionResult*]

6.3 Equilibrium

Nuclear statistical equilibrium (NSE) and equilibrium helpers.

This module is a pure-Python counterpart to the most commonly used libnuceq workflows. It solves for chemical potentials that reproduce a requested ρ , T and Y_e and then returns NSE abundances for the species that are present in a `nucnetpy.Network`.

The formulas are intentionally transparent and unit-aware. They are suitable for replacement workflows and regression testing, but exact equality with a specific compiled libnuceq build should be checked with the same nuclear masses, partition functions, Coulomb corrections, and numerical tolerances.

`nucnetpy.nse.nse_prefactor ( sp, t9, rho, include_partition=True )`

Return the NSE abundance prefactor excluding proton/neutron potentials.

The abundance is represented as $Y_i = \text{prefactor}_i * \exp(Z_i \mu_p / kT + N_i \mu_n / kT)$ where μ_p and μ_n are solved dimensionless chemical potential parameters in the internal solver. The mass excess term is included with the sign convention common in NSE abundance formulas.

Parameters

- **sp** (*Species*)
- **t9** (*float*)
- **rho** (*float*)
- **include_partition** (*bool*)

Return type *float*

class `nucnetpy.nse.NSEResult ( t9: 'float', rho: 'float', ye: 'float', mu_p: 'float', mu_n: 'float', abundances: 'Dict[str, float]', success: 'bool', message: 'str' = '' )`

Bases: *object*

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **mu_p** (*float*)
- **mu_n** (*float*)
- **abundances** (*Dict*[*str*, *float*])
- **success** (*bool*)
- **message** (*str*)

t9: *float*

rho: *float*

ye: *float*

```
mu_p: float
mu_n: float
abundances: Dict[str, float]
success: bool
message: str = ''
zone ( )
    Return type Zone
property xsum: float
property computed_ye: float
```

```
nucnetpy.nse.solve_nse ( network, t9, rho, ye, species=None,
include_partition=True, tol=1e-08, max_iter=200, nse_correction=None,
require_nuclear_data=True )
```

Solve an NSE composition for a network.

The constraints are $\sum(A_i Y_i) = 1$ and $\sum(Z_i Y_i) = Ye$. The solve works on well-scaled residuals with a numerically stable log-sum-exp, which keeps the iron-peak prefactors representable; see `_solve_equilibrium()` for the method. `success` is true only when the residual norm is below `tol`.

When every species in the solve has the same ratio N/Z , only the combination $Z \mu_p + N \mu_n$ is determined. The abundances are still unique, and the reported `mu_p` and `mu_n` are the minimum-norm pair, so they do not depend on the SciPy version.

`nse_correction` is an optional callable (`species, t9, rho, ye`) $\rightarrow$ `f_corr` added to each species' NSE exponent, the libnucnet NSE correction factor hook. Pass `nucnetpy.coulomb.nse_correction()` for the Bravo & Garcia-Senz Coulomb correction.

Parameters

- **network** ([Network](#))
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **species** (*Sequence[str] | None*)
- **include_partition** (*bool*)
- **tol** (*float*)
- **max_iter** (*int*)
- **require_nuclear_data** (*bool*)

Return type [NSEResult](#)

```
nucnetpy.nse.equilibrium_ratio ( reaction, network, t9, rho, ye )
```

Return product/reactant NSE abundance ratio for a reaction.

Parameters

- **network** ([Network](#))
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)

Return type float

Quasi-statistical equilibrium (QSE) with constrained clusters.

A pure-Python port of the libnuceq cluster workflow used by the NucNet Tools examples (`examples/analysis/compare_equil.cpp`) and the blog posts on cluster flows, (n,gamma)-(gamma,n) equilibrium, and comparing network calculations to equilibrium.

In libnuceq an equilibrium *cluster* is a subset of species (selected by an XPath expression such as `[z >= 6]`) whose total abundance is constrained to a prescribed value. Each cluster gets one extra chemical-potential multiplier `lambda_c` on top of the global proton/neutron potentials:

$$Y_i = \text{pref}_i * \exp(Z_i \mu_p + N_i \mu_n + \text{lambda}_{\{c(i)\}})$$

subject to $\sum A_i Y_i = 1$, $\sum Z_i Y_i = Y_e$, and $\sum_{\{i \text{ in } c\}} Y_i = Y_c$ for every cluster *c*. Full NSE is the special case with no clusters; an (n,gamma)-(gamma,n) r-process equilibrium is one cluster holding all heavy nuclei with `Y_c` equal to the current heavy-nuclei abundance.

Cluster membership is by species name; use `nucnetpy.network_limiter.select_species()` for XPath-like Z/A selection.

```
class nucnetpy.qse.QSECluster ( species, constraint, label='' )
```

Bases: object

A constrained equilibrium cluster: species names and their total Y.

Parameters

- **species** (*Sequence[str]*)
- **constraint** (*float*)
- **label** (*str*)

species: *Sequence[str]*

constraint: *float*

label: *str* = ''

```
class nucnetpy.qse.QSEResult ( t9: 'float', rho: 'float', ye: 'float', mu_p:
'float', mu_n: 'float', lambdas: 'List[float]', abundances: 'Dict[str, float]',
success: 'bool', message: 'str' = '' )
```

Bases: object

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **mu_p** (*float*)
- **mu_n** (*float*)
- **lambdas** (*List[float]*)
- **abundances** (*Dict[str, float]*)
- **success** (*bool*)
- **message** (*str*)

t9: *float*

rho: *float*

ye: *float*

mu_p: *float*

mu_n: *float*

```
lambdas: List[float]

abundances: Dict[str, float]

success: bool

message: str = ''

zone ( )

    Return type Zone

property xsum: float

property computed_ye: float
```

```
nucnetpy.qse.solve_qse ( network, t9, rho, ye, clusters, species=None,
include_partition=True, nse_correction=None, tol=1e-08, max_iter=300,
require_nuclear_data=True )
```

Solve a constrained (QSE) equilibrium for a network.

`clusters` is a sequence of *QSECluster*; species not in any cluster follow unconstrained NSE. A species may belong to at most one cluster. With `clusters=[]` the result equals `solve_nse()`, and the same solver is used, so `success` means the residual norm is below `tol` and non-unique potentials are reported as the minimum-norm solution. Species without nuclear data are left out, as in `solve_nse()`, unless `require_nuclear_data` is false.

Parameters

- **network** (*Network*)
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **clusters** (*Sequence[QSECluster]*)
- **species** (*Sequence[str] | None*)
- **include_partition** (*bool*)
- **tol** (*float*)
- **max_iter** (*int*)
- **require_nuclear_data** (*bool*)

Return type *QSEResult*

```
nucnetpy.qse.cluster_abundance ( abundances, cluster_species )
```

Return `Y_c`, the summed abundance of the cluster members.

Parameters

- **abundances** (*Mapping[str, float]*)
- **cluster_species** (*Sequence[str]*)

Return type *float*

```
nucnetpy.qse.cluster_ydot ( network, abundances, cluster_species, t9,
rho=1.0, use_reverse=False )
```

Return per-reaction contributions to dY_c/dt for a cluster.

Each reaction contributes `flux * (net change in cluster member count)`; reactions that only shuffle abundance inside the cluster contribute nothing (blog: “Calculating cluster flows”, C++ `compute_Ycdot`). With `use_reverse=True` the net (forward minus detailed-balance reverse) flux is used. `sum(result.values())` is dY_c/dt .

Parameters

- **network** (*Network*)

- **abundances** (*Mapping[str, float]*)
- **cluster_species** (*Sequence[str]*)
- **t9** (*float*)
- **rho** (*float*)
- **use_reverse** (*bool*)

Return type *Dict[str, float]*

Coulomb (plasma) corrections to NSE, ported from NucNet Tools.

This is a direct port of `user/nse_corr.cpp` (r647): the Coulomb correction of Bravo & Garcia-Senz (MNRAS 307, 984, 1999) with the fit constants of Ogata & Ichimaru (Phys. Rev. A 36, 5451, 1987) and the one-component-plasma value $f(\Gamma=1)/kT = -0.420$ of Slattery, Doolen & DeWitt (1980).

The species Coulomb chemical potential per kT is

$$\Gamma_i > 1: \mu_C/kT = a \Gamma + 4b \Gamma^{1/4} - 4c \Gamma^{-1/4} + d \ln \Gamma - o \quad \Gamma_i \leq 1: \mu_C/kT = (\beta/\gamma) \Gamma^\gamma - \Gamma^{3/2}/\sqrt{3}$$

with $G = \Gamma_i = Z^{5/3} \Gamma_e$ and the two branches matched in value and first derivative at $\Gamma_i = 1$. $\Gamma_e = e^2 / (a_e k T)$ uses the electron-cloud radius $a_e = (4 \pi \rho N_A Y_e / 3)^{-1/3}$.

The NSE correction factor is $f_{\text{corr}} = -\mu_C/kT$ (libnucnet convention): it is added to the exponent of the NSE abundance of each species and to the equilibrium constants used for detailed-balance reverse rates.

`nucnetpy.coulomb.gamma_e ( t9, rho, ye )`

Return the electron Coulomb coupling parameter Γ_e .

- Parameters**
- **t9** (*float*)
 - **rho** (*float*)
 - **ye** (*float*)

Return type *float*

`nucnetpy.coulomb.species_coulomb_chemical_potential ( z, t9, rho, ye )`

Return μ_C/kT for a species of charge z (0 for neutrons).

- Parameters**
- **z** (*int*)
 - **t9** (*float*)
 - **rho** (*float*)
 - **ye** (*float*)

Return type *float*

`nucnetpy.coulomb.species_coulomb_energy ( z, t9, rho, ye )`

Return the species Coulomb energy per particle in units of kT.

- Parameters**
- **z** (*int*)
 - **t9** (*float*)
 - **rho** (*float*)
 - **ye** (*float*)

Return type *float*

`nucnetpy.coulomb.species_coulomb_entropy ( z, t9, rho, ye )`

Return the species Coulomb entropy per particle in units of k_B .

Parameters

- **z** (*int*)
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)

Return type float

`nucnetpy.coulomb.nse_correction ( species, t9, rho, ye )`

Return the NSE correction factor $f_{\text{corr}} = -\mu_C/kT$ for a species.

Signature-compatible with the `nse_correction` hooks accepted by `nucnetpy.nse.solve_nse()` and the detailed-balance helpers: pass this function (or your own with the same signature) to enable Coulomb corrections.

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)

Return type float

`nucnetpy.coulomb.coulomb_entropy_per_nucleon ( abundances, species_map, t9, rho, ye )`

Return the total Coulomb entropy per nucleon in units of k_B .

$\sum_i Y_i \cdot s_C(Z_i)$ — the plasma part of the entropy used in the blog workflow “Computing NSE corrections to the entropy”.

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)

Return type float

6.4 Detailed balance

Reverse reaction rates from detailed balance.

This replaces the libnucnet behaviour of computing reverse rates from the forward rate, nuclear masses, and partition functions (blog workflows “Comparing forward and reverse reaction rates”, “Computing reaction flows”, and the (n,gamma)-(gamma,n) equilibrium studies).

For a reaction $\sum_r n_r R \rightarrow \sum_p n_p P$ that conserves Z and A , the equilibrium abundance ratio follows from the same Saha prefactors used by the NSE solver: $\ln K = \sum_p n_p \ln(\text{pref}_p) - \sum_r n_r \ln(\text{pref}_r)$ where $Y_i^{\text{eq}} = \text{pref}_i \cdot \exp(Z_i \mu_p + N_i \mu_n)$ and the chemical-potential terms cancel for a balanced reaction. Matching forward and reverse fluxes at equilibrium gives

$$\lambda_{\text{rev}} = \lambda_{\text{fwd}} \cdot \rho^{(n_r - n_p)} \cdot (s_{\text{rev}} / s_{\text{fwd}}) / K$$

with n the reactant/product orders and s the duplicate-particle statistical factors. Because $\ln(\text{pref})$ contains $-\ln(\rho \cdot N_A)$, the explicit density powers cancel and the reverse rate is a pure function of temperature, as it must be.

Photons and leptons (gamma, electron, ...) are ignored on either side, so the reverse of a radiative capture is the photodisintegration rate.

```
nucnetpy.detailed_balance.log_equilibrium_constant ( reaction,
species_map, t9, rho=1.0, include_partition=True )
```

Return $\ln K$ with $K = \prod Y_p^{\text{eq}} / \prod Y_r^{\text{eq}}$ for the reaction.

Requires mass excesses (and optionally partition functions) on the species; the Q-value enters through the mass excesses as $\exp(Q/kT)$.

Parameters

- **reaction** (*Reaction*)
- **species_map** (*Mapping[str, Species]*)
- **t9** (*float*)
- **rho** (*float*)
- **include_partition** (*bool*)

Return type float

```
nucnetpy.detailed_balance.reverse_rate ( reaction, species_map, t9,
rho=1.0, forward=None, include_partition=True )
```

Return the detailed-balance reverse rate for reaction at t_9 .

forward overrides the forward rate (default `reaction.rate(t9)`). The result is in the same convention as forward rates: to obtain a flux it must be combined with $\rho^{n_p - 1}$, product abundances, and the product-side statistical factor (see `net_flows()`).

Parameters

- **reaction** (*Reaction*)
- **species_map** (*Mapping[str, Species]*)
- **t9** (*float*)
- **rho** (*float*)
- **forward** (*float | None*)
- **include_partition** (*bool*)

Return type float

```
nucnetpy.detailed_balance.reverse_reaction ( reaction, species_map,
t9_grid=None, include_partition=True )
```

Return a *Reaction* for the reverse process with a tabulated rate.

The rate is sampled from `reverse_rate()` on *t9_grid* (default 30 points, $T_9 = 0.1 \dots 10$) so the reverse process — e.g. the (γ, n) partner of an (n, γ) capture — can be added to a network like any other reaction.

Parameters

- **reaction** (*Reaction*)
- **species_map** (*Mapping[str, Species]*)
- **t9_grid** (*Sequence[float] | None*)
- **include_partition** (*bool*)

Return type *Reaction*

```
nucnetpy.detailed_balance.consistent_reverse_network ( network,
t9_grid=None, include_partition=True, tabulate=False )
```

Return a copy of *network* whose reverse rates obey detailed balance.

Rate libraries such as JINA ReacLib supply the forward and reverse directions of a reaction as separately fitted expressions. Their ratio is therefore not exactly the equilibrium constant implied by the nuclear masses and partition functions in the accompanying nuclide file, and a network built from them relaxes to a stationary state that is not the nuclear statistical equilibrium of those same data. The discrepancy is not small: for the alpha chain at the fitted reverse rates differ from detailed balance by a median factor of 1.10, which displaces the equilibrium mass fractions by

a median of 4.5 per cent.

This function pairs every reaction with its inverse, keeps the exothermic member of each pair, and regenerates the endothermic member from `reverse_rate()`. Reactions with no inverse in the network are carried over unchanged. The resulting network relaxes to the composition returned by `nucnetpy.solve_nse()` to about one part in 10^6 , so the integrator and the equilibrium solver become mutually consistent.

By default the reverse rate is attached as a callable evaluated exactly at whatever temperature the integrator asks for. This matters: the reverse rate carries a factor $\exp(-E_a/RT)$, which is far too steep to interpolate accurately, and tabulating it on 200 logarithmically spaced points degrades the agreement with equilibrium from one part in 10^6 to about one part in 10^4 . Pass `tabulate=True` to sample the rate on `t9_grid` instead, which is less accurate but can be written to XML.

Note that this replaces measured or evaluated reverse rates with values derived from mass differences, so it enforces thermodynamic consistency rather than improving individual rates. Where a library reverse rate is known to be better than the masses it is derived from, keep the library value instead.

Parameters

- **t9_grid** (*Sequence[float] | None*)
- **include_partition** (*bool*)
- **tabulate** (*bool*)

```
nucnetpy.detailed_balance.reverse_flux ( reaction, species_map,
abundances, t9, rho=1.0, include_partition=True )
```

Return the detailed-balance reverse flux of `reaction` at this composition.

This is the flux the inverse reaction would carry with the rate from `reverse_rate()`, in the same units as `Reaction.flux()`.

Parameters

- **reaction** (*Reaction*)
- **species_map** (*Mapping[str, Species]*)
- **abundances** (*Mapping[str, float]*)
- **t9** (*float*)
- **rho** (*float*)
- **include_partition** (*bool*)

Return type float

```
nucnetpy.detailed_balance.net_flows ( network, abundances, t9, rho=1.0,
include_partition=True )
```

Return {`reaction`: (`forward`, `reverse`, `net`)} fluxes for a network.

The forward flux follows `Reaction.flux()`; the reverse flux uses the detailed-balance rate with the product abundances. At NSE abundances the net flux of every balanced reaction vanishes.

Every listed reaction is given its own detailed-balance reverse, including one whose inverse the network also lists explicitly, as JINA networks do. The entries for such a pair therefore describe the same process twice; do not add them up. `nucnetpy.analysis.entropy_generation_rate()` and `nucnetpy.analysis.integrated_currents()` handle pairs correctly.

Parameters

- **abundances** (*Mapping[str, float]*)
- **t9** (*float*)
- **rho** (*float*)
- **include_partition** (*bool*)

Return type *Dict[str, Tuple[float, float, float]]*

6.5 Screening, weak rates, decays

Electron-screening corrections for thermonuclear reaction rates.

The original NucNet Tools exposes user-defined screening callbacks. This module provides a Python callback-compatible implementation with common weak and intermediate-screening approximations. The API returns multiplicative factors applied to bare nuclear rates.

```
nucnetpy.screening.electron_number_density ( rho, ye )
```

Parameters

- **rho** (*float*)
- **ye** (*float*)

Return type float

```
nucnetpy.screening.debye_radius ( t9, rho, ye, ion_strength=0.0 )
```

Return Debye radius in cm.

ion_strength may be supplied as sum $Z_i^2 Y_i$. If omitted, the electron term alone is used, which is the usual safe weak-screening fallback.

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **ion_strength** (*float*)

Return type float

```
nucnetpy.screening.ion_strength ( abundances, species_map=None )
```

Parameters

- **abundances** (*Mapping[str, float]*)
- **species_map** (*Mapping[str, Species] | None*)

Return type float

```
class nucnetpy.screening.ScreeningContext ( t9: 'float', rho: 'float', ye:
'float', abundances: 'Mapping[str, float]', species_map: 'Optional[Map-
ping[str, Species]]' = None )
```

Bases: object

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **abundances** (*Mapping[str, float]*)
- **species_map** (*Mapping[str, Species] | None*)

t9: float

rho: float

ye: float

abundances: Mapping[str, float]

species_map: Mapping[str, Species] | None = None

property ion_strength: float

`nucnetpy.screening.weak_screening_factor ( z1, z2, t9, rho, ye, ion_strength_value=0.0 )`

Salpeter weak-screening enhancement factor.

The factor is $\exp(Z_1 Z_2 e^2 / (kT R_D))$. A cap prevents numerical overflow while retaining monotonic enhancement.

Parameters

- **z1** (*int*)
- **z2** (*int*)
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **ion_strength_value** (*float*)

Return type float

`nucnetpy.screening.graboske_intermediate_factor ( z1, z2, t9, rho, ye, ion_strength_value=0.0 )`

Smooth weak-to-intermediate approximation inspired by Graboske et al.

This is not tied to one compiled NucNet user file; it gives a stable, callback-compatible enhancement and can be replaced by a user function when exact project-specific screening is required.

Parameters

- **z1** (*int*)
- **z2** (*int*)
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **ion_strength_value** (*float*)

Return type float

`nucnetpy.screening.reaction_screening_factor ( reaction, context, model='weak' )`

Parameters

- **context** (*ScreeningContext*)
- **model** (*str*)

Return type float

class `nucnetpy.screening.SkyNetScreening ( species_map=None, max_z=110, pair_term=None )`

Bases: object

Screening as a Coulomb chemical potential per nuclear charge.

Instead of a pairwise enhancement factor, each charge Z is assigned a Coulomb chemical potential $\mu(Z)/kT$ in the plasma, and a reaction is enhanced by

$$\exp(\text{sum_reactants } \mu(Z_i)/kT - \mu(\text{sum_i } Z_i)/kT),$$

the difference between the potentials of the separated reactants and of the fused compound charge. This has two advantages over a pairwise factor: it extends naturally to reactions with more than two charged reactants, and because it is a chemical potential it can be applied to the forward and reverse directions consistently, so screening does not break detailed balance.

$\mu(Z)$ blends three regimes with smooth tanh switches: weak screening, $\mu_w = -Z^2 \text{ zeta_lambda_0} / 2$; intermediate screening, $\mu_i = -0.380 \text{ lambda_0}^b \text{ eta_b } Z^{(b+1)}$

with $b = 0.860$; and strong screening from a one-component-plasma fit.

ζ is the Debye sum per particle, $\zeta^2 = (\sum Z^2 Y + E) / \sum Y$, where E is the electrons' and positrons' share. As in SkyNet, E comes from the electron gas: by default (`pair_term=None`) it is computed at every update by `nucnetpy.electrons` as $kT (dn/d\mu) / (\rho N_A)$. That is Y_e for non-degenerate electrons, which recovers Salpeter's $\zeta^2 = \sum (Z^2 + Z) Y$; it falls towards zero as the electrons become degenerate and no longer screen, and it rises above Y_e when positron pairs add to the screening. A number passed as `pair_term` is used instead, so `pair_term=0.0` gives the ions-only (fully degenerate) limit.

The electron state is cached and only recomputed when the temperature, density or Y_e change (Y_e is compared to 10 decimal places).

The object is a callable suitable for the screening argument of `nucnetpy.evolve_zone()`. It caches $\mu(Z)$ and recomputes it in `update()`, which the network calls once per right-hand-side evaluation, so the cost does not scale with the number of reactions.

Parameters

- **species_map** (*Optional*[*Mapping*[*str*, *Species*]])
- **max_z** (*int*)
- **pair_term** (*Optional*[*float*])

electron_term (*t9*, *rho*, *ye*)

Return E , the electrons' and positrons' share of the Debye sum.

This is `pair_term` if one was given, and otherwise `nucnetpy.electrons.ElectronGas.screening_term` at (*t9*, *rho*, *ye*).

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)

Return type float

update (*abundances*, *t9*, *rho*, *ye=None*)

Recompute $\mu(Z)$ for this composition and thermodynamic state.

Parameters

- **abundances** (*Mapping*[*str*, *float*])
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float* | *None*)

Return type None

chemical_potential (*z*)

Return $\mu(Z)/kT$ for charge z .

Parameters **z** (*int*)

Return type float

factor (*reaction*)

Return the multiplicative screening enhancement of *reaction*.

Return type float

Weak-rate and neutrino helper layer.

Supports two-dimensional $T9/\rho Y_e$ tables, simple FFN/Oda/Langanke-style text blocks after normalization, bilinear interpolation, and ODE source-term hooks.

```
class nucnetpy.weak.WeakRateTable ( parent: 'str', daughter: 'str', t9:
'Sequence[float]', rho_je: 'Sequence[float]', rates: 'np.ndarray', nu_loss:
'Optional[np.ndarray]' = None, label: 'str' = '' )
```

Bases: object

Parameters

- **parent** (*str*)
- **daughter** (*str*)
- **t9** (*Sequence[float]*)
- **rho_je** (*Sequence[float]*)
- **rates** (*ndarray*)
- **nu_loss** (*ndarray | None*)
- **label** (*str*)

parent: *str*

daughter: *str*

t9: *Sequence[float]*

rho_je: *Sequence[float]*

rates: *ndarray*

nu_loss: *ndarray | None = None*

label: *str = ''*

rate (*t9, rho_je*)

Parameters

- **t9** (*float*)
- **rho_je** (*float*)

Return type *float*

neutrino_loss (*t9, rho_je*)

Parameters

- **t9** (*float*)
- **rho_je** (*float*)

Return type *float*

nucnetpy.weak.**read_weak_table** (*path, parent=None, daughter=None*)

Read a normalised weak-rate table.

Accepted format:

```
# parent fe56 daughter mn56
t9 rhoYe rate [nu_loss]
1.0 1e7 2.3e-4 1.0e-5
```

Repeated rows are reshaped into a rectangular T9 x rhoYe grid.

Parameters

- **path** (*str | Path*)
- **parent** (*str | None*)
- **daughter** (*str | None*)

Return type *WeakRateTable*

```
nucnetpy.weak.read_weak_tables ( paths )
```

Parameters `paths` (`Iterable[str | Path]`)

Return type `List[WeakRateTable]`

```
nucnetpy.weak.compute_yedot ( abundances, weak_rates, species_map, t9,
rho, ye )
```

Parameters

- `abundances` (`Mapping[str, float]`)
- `weak_rates` (`Sequence[WeakRateTable]`)
- `t9` (`float`)
- `rho` (`float`)
- `ye` (`float`)

Return type `float`

```
nucnetpy.weak.weak_source_terms ( abundances, weak_rates, t9, rho, ye )
```

Parameters

- `abundances` (`Mapping[str, float]`)
- `weak_rates` (`Sequence[WeakRateTable]`)
- `t9` (`float`)
- `rho` (`float`)
- `ye` (`float`)

Return type `Dict[str, float]`

Nuclear decay helpers ported into Python.

```
class nucnetpy.decay.DecayRecord ( parent: 'str', daughter: 'str',
half_life: 'float', branch: 'float' = 1.0, mode: 'str' = 'decay' )
```

Bases: `object`

Parameters

- `parent` (`str`)
- `daughter` (`str`)
- `half_life` (`float`)
- `branch` (`float`)
- `mode` (`str`)

`parent: str`

`daughter: str`

`half_life: float`

`branch: float = 1.0`

`mode: str = 'decay'`

`property rate: float`

`reaction ( )`

Return type `Reaction`

```
nucnetpy.decay.add_decay_records ( network, records )
```

Parameters `records` (`Iterable[DecayRecord]`)

Return type `None`

```
nucnetpy.decay.decay_constant_from_half_life ( half_life )
```

Parameters `half_life` (`float`)

Return type `float`

```
nucnetpy.decay.fission_reaction ( parent, fragments, neutrons=0, rate=0.0,
mode='sf' )
```

Return a fission reaction `parent -> fragments + neutrons * n`.

`rate` is the fission decay constant in 1/s (use `decay_constant_from_half_life()` for spontaneous-fission half-lives). `mode` labels the channel, e.g. "sf" (spontaneous) or "bdf" (beta-delayed); for neutron-induced fission build a normal two-reactant Reaction instead. Blog workflow: "Adding fission to an r-process calculation".

Parameters

- `parent` (`str`)
- `fragments` (`Iterable[str]`)
- `neutrons` (`int`)
- `rate` (`float`)
- `mode` (`str`)

Return type `Reaction`

Neutrino reaction-rate helper functions.

```
class nucnetpy.neutrino.NeutrinoLuminosity ( l0: 'float', tau: 'float |
None' = None )
```

Bases: `object`

Parameters

- `l0` (`float`)
- `tau` (`float | None`)

`l0`: `float`

`tau`: `float | None = None`

`luminosity` (`t`)

Parameters `t` (`float`)

Return type `float`

```
class nucnetpy.neutrino.NeutrinoQuantity ( temperature: 'Sequence[float]',
log10_xsec: 'Sequence[float]' )
```

Bases: `object`

Parameters

- `temperature` (`Sequence[float]`)
- `log10_xsec` (`Sequence[float]`)

`temperature`: `Sequence[float]`

`log10_xsec`: `Sequence[float]`

`cross_section` (`tnu`)

Parameters `tnu` (`float`)

Return type float

```
nucnetpy.neutrino.geometric_flux_rate ( luminosity, mean_energy_mev,
radius_cm, cross_section_cm2 )
```

Parameters

- **luminosity** (*float*)
- **mean_energy_mev** (*float*)
- **radius_cm** (*float*)
- **cross_section_cm2** (*float*)

Return type float

6.6 Analysis

Analysis helpers equivalent to common NucNet Tools example programs.

```
nucnetpy.analysis.largest_mass_fractions ( zone, species_map, n=10,
min_x=0.0 )
```

Parameters

- **zone** (*Zone*)
- **species_map** (*Mapping[str, Species]*)
- **n** (*int*)
- **min_x** (*float*)

```
nucnetpy.analysis.abundances_for_element ( zone, element, species_map )
```

Parameters

- **zone** (*Zone*)
- **element** (*str*)
- **species_map** (*Mapping[str, Species]*)

Return type *Dict[str, float]*

```
nucnetpy.analysis.abundances_vs_nucleon_number ( zone, species_map )
```

Parameters

- **zone** (*Zone*)
- **species_map** (*Mapping[str, Species]*)

Return type *Dict[int, float]*

```
nucnetpy.analysis.element_abundances ( zone, species_map )
```

Parameters

- **zone** (*Zone*)
- **species_map** (*Mapping[str, Species]*)

Return type *Dict[str, float]*

```
nucnetpy.analysis.abundance_moment ( zone, species_map, moment=1, kind='a'
)
```

Parameters

- **zone** (*Zone*)
- **species_map** (*Mapping[str, Species]*)
- **moment** (*int*)

- **kind**(*str*)

Return type float

nucnetpy.analysis.**species_history** (*network*, *species*)

- Parameters
- **network** (*Network*)
 - **species** (*str*)

Return type *List[Tuple[int, Tuple[str, str, str], float]]*

nucnetpy.analysis.**flows** (*network*, *zone_index*=0, *t9*=None, *rho*=None)

- Parameters
- **network** (*Network*)
 - **zone_index** (*int*)
 - **t9** (*float* | *None*)
 - **rho** (*float* | *None*)

nucnetpy.analysis.**ydot** (*network*, *zone_index*=0, *t9*=None, *rho*=None)

- Parameters
- **network** (*Network*)
 - **zone_index** (*int*)
 - **t9** (*float* | *None*)
 - **rho** (*float* | *None*)

nucnetpy.analysis.**energy_generation_rate** (*network*, *zone_index*=0, *t9*=None, *rho*=None)

- Parameters
- **network** (*Network*)
 - **zone_index** (*int*)
 - **t9** (*float* | *None*)
 - **rho** (*float* | *None*)

Return type float

nucnetpy.analysis.**compare_rates** (*net_a*, *net_b*, *t9*, *rho*=1.0)

- Parameters
- **net_a** (*Network*)
 - **net_b** (*Network*)
 - **t9** (*float*)
 - **rho** (*float*)

Return type *List[Tuple[str, float, float, float]]*

nucnetpy.analysis.**separation_energy** (*species_name*, *species_map*, *particle*='n')

Return S_n or S_p in MeV, or None if the needed nuclides are absent.

$S_n(Z, A) = ME(Z, A-1) + ME(n) - ME(Z, A)$ and $S_p(Z, A) = ME(Z-1, A-1) + ME(p) - ME(Z, A)$.

- Parameters
- **species_name** (*str*)
 - **species_map** (*Mapping[str, Species]*)
 - **particle** (*str*)

Return type float | None

```
nucnetpy.analysis.charge_changing_flows ( network, zone_index=0, t9=None,
rho=None )
```

Return each reaction's contribution to dY_e/dt (flux times net ΔZ).

Only reactions with a nonzero nuclear charge change (weak reactions such as beta decays and electron captures) appear in the result; strong reactions conserve Z and contribute nothing. `sum(result.values())` is dY_e/dt .

Parameters

- **network** (*Network*)
- **zone_index** (*int*)
- **t9** (*float* | *None*)
- **rho** (*float* | *None*)

Return type *Dict*[str, float]

```
nucnetpy.analysis.system_timescales ( network, zone_index=0, t9=None,
rho=None )
```

Return per-species timescales $Y / |dY/dt|$ in seconds.

Species with zero derivative get *inf*. The shortest timescales identify the stiffest components of the system (blog: "Computing system timescales").

Parameters

- **network** (*Network*)
- **zone_index** (*int*)
- **t9** (*float* | *None*)
- **rho** (*float* | *None*)

Return type *Dict*[str, float]

```
nucnetpy.analysis.heavy_nuclei_abundance ( zone, species_map, zmin=3 )
```

Return Y_h , the total abundance of nuclei with $Z \geq zmin$.

The photon-to-heavy-nucleus ratio and the r-process neutron-to-seed ratio are built from this quantity (blog: "Computing the number of heavy nuclei").

Parameters

- **zone** (*Zone*)
- **species_map** (*Mapping*[str, *Species*])
- **zmin** (*int*)

Return type float

```
nucnetpy.analysis.neutron_exposure ( result, thermo )
```

Return the s-process neutron exposure $\tau = \int n_n v_T dt$ in mb^{-1} .

`result` is an *EvolutionResult* whose species include the neutron; `thermo` is the same (`t`, abundances) $\rightarrow$ (`t9`, `rho`) function used for the evolution. $n_n = \rho N_A Y_n$ and the thermal velocity is $v_T = \sqrt{2 k T / m_n}$ (blog: "Computing the s-process neutron exposure").

Return type float

```
nucnetpy.analysis.reaction_entropy_changes ( network, abundances, t9, rho
)
```

Return per-reaction entropy change ΔS in units of k_B per reaction.

$\Delta S = -\sum_i n_{u_i} \mu_{u_i}/kT$ with Maxwell-Boltzmann chemical potentials $\mu_{u_i}/kT = \ln Y_{u_i} - \ln \text{pref}_{u_i}$; this equals the C++ per-reaction $Q/kT + \sum_r \ln(Y/Y_Q) - \sum_p \ln(Y/Y_Q)$ of NucNet Tools `flow_utilities.cpp` and, under detailed balance, $\ln(f/r)$.

ΔS is formally infinite for a reaction that involves an absent species, because $\ln Y$ diverges as Y

tends to zero. Such an abundance is replaced by $1e-300$ here, so the value returned for that reaction reflects the floor rather than the physics; `entropy_generation_rate()` leaves those reactions out.

Parameters

- **network** (*Network*)
- **abundances** (*Mapping[str, float]*)
- **t9** (*float*)
- **rho** (*float*)

Return type *Dict[str, float]*

```
nucnetpy.analysis.entropy_generation_rate ( network, zone_index=0,
t9=None, rho=None, use_reverse=True )
```

Return dS/dt per nucleon in units of k_B per second.

Ports NucNet Tools `compute_entropy_generation_rate`: the sum over reactions of $(f - r) * \Delta S$, where ΔS is the per-reaction entropy change of `reaction_entropy_changes()` (blog series “Computing the entropy generation rate”). Each forward/reverse pair is counted once. When the network lists the inverse of a reaction, as JINA networks do, r is that inverse’s own flux, so the result is exactly $-\sum_i (\mu_i/kT) dY_i/dt$ for the network being evolved. Otherwise r is the detailed-balance reverse flux, as in NucNet Tools. With reverse rates that obey detailed balance every term equals $(f - r) \ln(f/r)$, so the total is non-negative and vanishes at NSE.

With `use_reverse=False` only the listed reactions are used, each with its forward flux: $\sum_r f_r \Delta S_r = -\sum_i (\mu_i/kT) dY_i/dt$ for exactly the network as given, with no detailed-balance reverses implied.

Reactions that involve an absent species ($Y \leq 1e-100$, the NucNet Tools threshold) are left out. Creating a species from zero abundance produces formally infinite entropy, so any finite value for such a reaction would be set by an arbitrary floor rather than by the physics; treat the total for a composition with absent species as a lower bound. (NucNet Tools skips a reaction when its forward flux vanishes, and drops the logarithm of an absent product, which can give a term of the wrong sign.)

Electron and neutrino chemical-potential terms of the C++ version are not included; nucnetpy networks carry weak reactions with their own tabulated rates instead.

Parameters

- **network** (*Network*)
- **zone_index** (*int*)
- **t9** (*float | None*)
- **rho** (*float | None*)
- **use_reverse** (*bool*)

Return type *float*

```
nucnetpy.analysis.integrated_currents ( network, result, thermo,
use_reverse=True )
```

Return per-reaction time-integrated net currents over an evolution.

Ports NucNet Tools `update_flow_currents`: for each reaction the current accumulates $(\text{forward} - \text{reverse}) * dt$ across the time grid of `result` (an `EvolutionResult`), integrated here with the trapezoidal rule. The integrated current of a reaction is the net number of transitions per nucleon it produced over the calculation (blog: “Creating integrated currents diagrams”, “Analyzing integrated currents quantitatively”).

The reverse flux is the flux of the inverse reaction when the network lists it, and the detailed-balance value otherwise. A pair listed in both directions therefore appears twice, with equal and opposite currents. With `use_reverse=False` each entry is the reaction’s own forward current. Reactions listed more than once are added together.

Parameters

- **network** (*Network*)

- **use_reverse** (*bool*)

Return type *Dict*[*str*, *float*]

`nucnetpy.analysis.nuclear_energy_generation_rate ( network, zone_index=0, t9=None, rho=None, screening=None, abundances=None, ydot_values=None )`

Return the nuclear energy generation rate in erg per gram per second.

The rate follows from the change in total mass excess of the composition,

$$\text{eps} = -N_A * C * \sum_i (dY_i/dt) * \text{Delta } M_i,$$

with `Delta M_i` the mass excess in MeV and `C = 1.602176634e-6 erg/MeV` (`constants.MEV_TO_ERG`) converting the sum, which carries units of MeV/g/s, into erg/g/s. This is exact, and it is preferable to summing `Q_r` times the flow of each reaction: it does not use the reaction `Q`-values stored in the rate library, so it is internally consistent with the adopted mass-excess table – the same table the equilibrium solver uses – and energy release and equilibrium are therefore built from one set of nuclear data. It may differ from library `Q`-values when the two data sources use different nuclear masses.

A positive value denotes energy released to the plasma. Neutrino losses are not subtracted: energy carried away by neutrinos from weak reactions must be accounted for separately.

`abundances` and `ydot_values` override the zone composition and its derivative, which is useful when the rate is wanted along a trajectory that has already been integrated.

Parameters

- **network** (*Network*)
- **zone_index** (*int*)
- **t9** (*float* | *None*)
- **rho** (*float* | *None*)
- **abundances** (*Mapping*[*str*, *float*] | *None*)
- **ydot_values** (*Mapping*[*str*, *float*] | *None*)

Return type *float*

`nucnetpy.analysis.nuclear_energy_release ( network, initial, final )`

Return the energy released between two compositions, in erg per gram.

This is the time integral of `nuclear_energy_generation_rate()` and depends only on the endpoints, being the difference in total mass excess.

Parameters

- **network** (*Network*)
- **initial** (*Mapping*[*str*, *float*])
- **final** (*Mapping*[*str*, *float*])

Return type *float*

6.7 Input and output

Fast, permissive NucNet/JINA/libnucnet XML reader and writer.

This module is tuned for real JINA XML databases, where files may contain thousands of nuclides and tens of thousands of reactions. It supports:

- separate `<nuclear_data>` and `<reaction_data>` files,
- combined `<libnucnet_input>` output files,
- zone files with mass fractions or abundances,
- ReacLib/JINA `non_smoker_fit` blocks with `a1–a7` coefficients,

- single-rate beta-decay style records,
- partition-function tables.

`nucnetpy.io.xml.read_xml ( path )`

Parameters `path` (*str* | *Path*)

Return type *Network*

`nucnetpy.io.xml.read_network_xml ( path )`

Parameters `path` (*str* | *Path*)

Return type *Network*

`nucnetpy.io.xml.read_xml_root ( root, source='<xml-root>' )`

Parameters `source` (*str*)

Return type *Network*

`nucnetpy.io.xml.write_xml ( network, path )`

Parameters

- `network` (*Network*)
- `path` (*str* | *Path*)

Return type *None*

`nucnetpy.io.xml.write_zone_xml ( network, path )`

Parameters

- `network` (*Network*)
- `path` (*str* | *Path*)

Return type *None*

`nucnetpy.io.xml.read_xml_string ( text )`

Parameters `text` (*str*)

Return type *Network*

JINA/libnucnet XML database helpers.

The original NucNet Tools workflow usually reads nuclide data and reaction data from JINA ReacLib /libnucnet XML files. These helpers keep that workflow in the pure-Python port: one file may contain nuclides, another may contain reactions, and both can be merged into one `nucnetpy.core.Network` object.

`nucnetpy.io.jina.read_jina_xml ( nuclides_xml, reactions_xml, zones_xml=None )`

Read separate JINA/libnucnet nuclide and reaction XML files.

Parameters

- `nuclides_xml` (*str* | *Path*) – XML file containing `<nuclide>` records with *Z*, *A*, mass excess, spin, and optional partition-function data.
- `reactions_xml` (*str* | *Path*) – XML file containing `<reaction>` records and ReacLib rate fits.
- `zones_xml` (*str* | *Path* | *None*) – Optional XML file containing zones /abundances. If omitted, the returned network contains species and reactions but no zones.

Returns A single Python network object with species, reactions, and optionally zones.

Return type *Network*

```
nucnetpy.io.jina.combine_jina_xml ( nuclides_xml, reactions_xml,
output_xml, zones_xml=None )
```

Read separate JINA XML files and write one combined nucnetpy XML file.

Parameters

- **nuclides_xml** (*str* | *Path*)
- **reactions_xml** (*str* | *Path*)
- **output_xml** (*str* | *Path*)
- **zones_xml** (*str* | *Path* | *None*)

Return type *Network*

```
nucnetpy.io.jina.jina_database_summary ( net_or_nuclides,
reactions_xml=None, zones_xml=None )
```

Return a compact summary of a loaded JINA database.

Accepts either an already loaded Network, or file paths (nuclides_xml, reactions_xml[, zones_xml]).

Return type dict[str, int]

6.8 Utilities

Network limiting utilities for reduced-network calculations.

```
nucnetpy.network_limiter.select_species ( network, zmin=None, zmax=None,
amin=None, amax=None, elements=None )
```

Return species names matching Z/A ranges and/or element symbols.

A pure-Python replacement for the libnucnet nuclide XPath selections used in the blog workflows “Selecting an input network” and “Applying a nuclide XPath expression”. Combine with `limit_network()`:

```
limit_network(net, select_species(net, zmin=1, zmax=28, amax=60))
```

Parameters

- **zmin** (*int* | *None*)
- **zmax** (*int* | *None*)
- **amin** (*int* | *None*)
- **amax** (*int* | *None*)
- **elements** (*Iterable*[*str*] | *None*)

Return type *List*[*str*]

```
nucnetpy.network_limiter.limit_network ( network, species,
include_linked=False )
```

Restrict network in place to the given nuclides and the reactions among them.

A reaction survives if every nuclide it involves is kept. Photons and leptons are not nuclides, so they never decide the outcome: a photodisintegration among kept nuclides survives whether or not "gamma" is listed, and listing it is harmless. With `include_linked` the kept set grows to every nuclide that shares a reaction with it; the photon does not count as a link, or every photon reaction would join the network.

Parameters

- **species** (*Iterable*[*str*])
- **include_linked** (*bool*)

Hydrodynamic trajectory utilities replacing user/hydro* helpers.

```
class nucnetpy.hydro.Trajectory ( time: 'np.ndarray', t9: 'np.ndarray',  
rho: 'np.ndarray' )
```

Bases: object

Parameters

- **time** (ndarray)
- **t9** (ndarray)
- **rho** (ndarray)

time: ndarray

t9: ndarray

rho: ndarray

thermo (t, y=None)

Parameters **t** (float)

Return type *Tuple*[float, float]

classmethod **from_columns** (time, t9, rho)

Parameters

- **time** (*Sequence*[float])
- **t9** (*Sequence*[float])
- **rho** (*Sequence*[float])

Return type *Trajectory*

nucnetpy.hydro.**read_trajectory** (path)

Parameters **path** (str | Path)

Return type *Trajectory*

nucnetpy.hydro.**exponential_expansion** (t0, t9_0, rho_0, tau, n=200,
t_end=None)

Parameters

- **t0** (float)
- **t9_0** (float)
- **rho_0** (float)
- **tau** (float)
- **n** (int)
- **t_end** (float | None)

Return type *Trajectory*

Thermodynamics of the burning matter: entropy, and finding T or rho from it.

`entropy_per_nucleon()` adds up the entropy of the three parts of the matter, each treated in full:

- **ions** - the Sackur-Tetrode entropy of each nuclear species, with its real mass, ground-state spin and temperature-dependent partition function;
- **electrons and positrons** - from `nucnetpy.electrons`, at any degeneracy and including pairs;
- **photons** - black-body radiation, $4 \pi^2 / (15 \rho N_A k)$;

and, optionally, the Coulomb (plasma) correction of `nucnetpy.coulomb`.

`density_for_entropy()` and `t9_for_entropy()` search for the density or temperature that

gives a target entropy at fixed composition, and `constant_entropy_thermo()` uses the second of these to drive a network calculation along a constant-entropy (adiabatic) expansion.

All entropies are per nucleon in units of Boltzmann's constant.

`nucnetpy.thermo.ideal_gas_pressure ( rho, temperature, ytot )`

Parameters

- `rho (float)`
- `temperature (float)`
- `ytot (float)`

Return type float

`nucnetpy.thermo.radiation_pressure ( temperature )`

Parameters `temperature (float)`

Return type float

class `nucnetpy.thermo.EntropyPerNucleon ( ions, electrons, photons, coulomb=0.0 )`

Bases: object

The entropy per nucleon, in units of k_B , split into its parts.

Parameters

- `ions (float)`
- `electrons (float)`
- `photons (float)`
- `coulomb (float)`

ions: float

electrons: float

photons: float

coulomb: float = 0.0

property total: float

`nucnetpy.thermo.photon_entropy_per_nucleon ( t9, rho )`

Entropy of black-body radiation per nucleon, $4 a T^3 / (3 \rho N_A k)$.

Parameters

- `t9 (float)`
- `rho (float)`

Return type float

`nucnetpy.thermo.ion_entropy_per_nucleon ( abundances, species, t9, rho, include_partition=True )`

Sackur-Tetrode entropy of the nuclei per nucleon, in units of k_B .

$\sum_i Y_i (5/2 + \ln(Y_{Q,i} / Y_i) + d \ln G_i / d \ln T)$, where $Y_{Q,i} = (2J_i+1) G_i(T) n_{Q,i} / (\rho N_A)$ uses the nuclear mass through the quantum concentration $n_{Q,i} = (A_i m_u kT / 2 \pi \hbar^2)^{3/2}$. The last term is the entropy of the nuclei's excited states. Species with zero abundance and massless particles contribute nothing.

Parameters

- `abundances (Mapping[str, float])`
- `t9 (float)`
- `rho (float)`

- **include_partition** (*bool*)

Return type float

```
nucnetpy.thermo.entropy_per_nucleon ( abundances, species, t9, rho,  
ye=None, include_partition=True, electrons=True, photons=True,  
coulomb=False )
```

Return the entropy per nucleon of the matter, split into its parts.

abundances maps species names to abundances Y ; *species* is a `Network` or a mapping of names to `Species`, which supplies the spins and partition functions (species missing from it are parsed from their names and get a weight of one). *ye* defaults to $\sum Z Y$ of the abundances. *t9* is the temperature in 10^9 K and *rho* the density in g cm^{-3} .

The flags choose which parts are computed; parts that are switched off are reported as zero. The Coulomb correction is off by default, as in the NSE solver.

Example: pure helium-4 at $T_9 = 5$ and $\rho = 1e8$ has ion entropy 3.24, electron entropy 1.07 and photon entropy 0.15 per nucleon.

Parameters

- **abundances** (*Mapping[str, float]*)
- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float | None*)
- **include_partition** (*bool*)
- **electrons** (*bool*)
- **photons** (*bool*)
- **coulomb** (*bool*)

Return type *EntropyPerNucleon*

```
nucnetpy.thermo.density_for_entropy ( entropy, abundances, species, t9,  
ye=None, bounds=(1e-12, 1000000000000000.0), include_partition=True,  
electrons=True, photons=True, coulomb=False )
```

Return the density (g cm^{-3}) at which the matter has the given entropy.

The composition and the temperature *t9* are held fixed, and the entropy is `entropy_per_nucleon()` with the same options. The entropy falls steadily as the density rises, so the answer is unique; it is found by a bracketing search in $\ln \rho$ between *bounds*, to a relative precision of about 10^{-14} . A `ValueError` reports the entropy range those bounds can reach if the target is outside it.

Parameters

- **entropy** (*float*)
- **abundances** (*Mapping[str, float]*)
- **t9** (*float*)
- **ye** (*float | None*)
- **bounds** (*Tuple[float, float]*)
- **include_partition** (*bool*)
- **electrons** (*bool*)
- **photons** (*bool*)
- **coulomb** (*bool*)

Return type float


```
nucnetpy.thermo.t9_for_entropy ( entropy, abundances, species, rho,
ye=None, bounds=(0.001, 100.0), include_partition=True, electrons=True,
photons=True, coulomb=False )
```

Return the temperature (in 10^9 K) at which the matter has the given entropy.

The composition and the density ρ are held fixed. The entropy rises with temperature, and the search runs in $\ln T_9$ between bounds; see `density_for_entropy()`.

Parameters

- **entropy** (*float*)
- **abundances** (*Mapping[str, float]*)
- **rho** (*float*)
- **ye** (*float | None*)
- **bounds** (*Tuple[float, float]*)
- **include_partition** (*bool*)
- **electrons** (*bool*)
- **photons** (*bool*)
- **coulomb** (*bool*)

Return type *float*

```
nucnetpy.thermo.constant_entropy_thermo ( entropy, density, species,
ye=None, bounds=(0.001, 100.0), include_partition=True, electrons=True,
photons=True, coulomb=False )
```

Return a thermodynamic callable for a constant-entropy expansion.

`density(t)` gives the density at time t . Each time the network asks for the conditions, the temperature is set so that the matter, with its composition at that moment, has the given entropy per nucleon. Pass the result as the `thermo` argument of `nucnetpy.evolve_zone()`. This is the constant-entropy evolution of the NucNet Tools blog. The entropy is held fixed, so the heat released by the reactions is not fed back; that is a good approximation once the burning has slowed down, as in the freeze-out from a hot, expanding plasma. `ye` defaults to the composition's own $\sum Z_i Y_i$ at each call.

The search starts from the previous temperature, so following a smooth expansion costs a few entropy evaluations per call.

Parameters

- **entropy** (*float*)
- **density** (*Callable[[float], float]*)
- **ye** (*float | None*)
- **bounds** (*Tuple[float, float]*)
- **include_partition** (*bool*)
- **electrons** (*bool*)
- **photons** (*bool*)
- **coulomb** (*bool*)

```
nucnetpy.thermo.entropy_ideal_ions ( rho, temperature, ytot )
```

Deprecated: return $Y_{\text{tot}} * (5/2 + \ln(T^{1.5} / \rho))$, a scale-free proxy.

This is not the physical entropy: it has no nuclear masses, partition functions or physical constants, and no electrons or photons. For pure helium-4 at $T = 5e9$ K and $\rho = 1e8$ g/cm³ it gives 4.39 against a true ion entropy of 3.24. Use `entropy_per_nucleon()` instead. temperature is in kelvin.

Parameters

- **rho** (*float*)
- **temperature** (*float*)

- **ytot** (*float*)

Return type float

`nucnetpy.thermo.density_from_entropy ( entropy, temperature, ytot )`

Deprecated: invert `entropy_ideal_ions()` for the density.

A physical entropy does not give a physical density here: inverting the true entropy of pure helium-4 at $\rho = 1e8 \text{ g/cm}^3$ returns about $1e10$. Use `density_for_entropy()` instead.

- Parameters
- **entropy** (*float*)
 - **temperature** (*float*)
 - **ytot** (*float*)

Return type float

`nucnetpy.thermo.temperature_from_entropy ( entropy, rho, ytot )`

Deprecated: invert `entropy_ideal_ions()` for the temperature, in kelvin.

Use `t9_for_entropy()` instead.

- Parameters
- **entropy** (*float*)
 - **rho** (*float*)
 - **ytot** (*float*)

Return type float

`nucnetpy.thermo.sound_speed_ideal_gamma ( rho, pressure, gamma=1.6666666666666667 )`

- Parameters
- **rho** (*float*)
 - **pressure** (*float*)
 - **gamma** (*float*)

Return type float

`nucnetpy.thermo.q_energy_erg_per_g ( q_mev )`

- Parameters
- **q_mev** (*float*)

Return type float

The electron-positron gas: chemical potential, pressure, energy, entropy.

NucNet Tools takes the properties of the electrons from `libstatmech`. This module is the pure-Python counterpart. Given the temperature, the density and the electron fraction Y_e it finds the electron chemical potential and, from it, everything else the rest of the package needs:

- the pressure, energy and entropy of the electrons and positrons, used by `nucnetpy.thermo.entropy_per_nucleon()`;
- the electrons' contribution to Debye screening, used by `nucnetpy.screening.SkyNetScreening`.

The gas is treated exactly: the electrons may be relativistic and degenerate to any degree, and the positrons that appear in hot, thin matter are included. Their chemical potential is the negative of the electrons' (the reaction $e^- + e^+ \leftrightarrow \text{photons}$ is in equilibrium), and charge neutrality fixes the net number of electrons per nucleon to Y_e .

Notation. $\beta = kT / (m_e c^2)$ and η is the electron chemical potential without the rest mass, in units of kT . The positrons then have $\eta_+ = -\eta - 2/\beta$. All quantities come from the integrals

$I = \text{integral over } x \text{ of } (\text{powers of } x \text{ and } (1 + \beta x)) / (\exp(x - \eta) + 1)$

where x is the kinetic energy in units of kT . They are evaluated by Gauss-Legendre quadrature in $t = \sqrt{x}$, which removes the square-root behaviour at $x = 0$, with the intervals packed around the Fermi edge at $x = \eta$. The quadrature agrees with SciPy's adaptive integration to about one part in 10^{12} (see the tests).

```
class nucnetpy.electrons.ElectronGas ( t9, rho, ye, eta, beta,
n_electrons, n_positrons, pressure, energy, entropy_density, dn_deta )
```

Bases: object

The state of the electron-positron gas at one ($T9$, ρ , Y_e).

Densities are in cm^{-3} , the pressure and energy densities in erg cm^{-3} and the entropy density in units of k_B per cm^3 . The energies are kinetic energies; the rest mass of the positron pairs is in `pair_rest_energy`.

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)
- **eta** (*float*)
- **beta** (*float*)
- **n_electrons** (*float*)
- **n_positrons** (*float*)
- **pressure** (*float*)
- **energy** (*float*)
- **entropy_density** (*float*)
- **dn_deta** (*float*)

t9: float

rho: float

ye: float

eta: float

beta: float

n_electrons: float

n_positrons: float

pressure: float

energy: float

entropy_density: float

dn_deta: float

property mu_mev: float

Electron chemical potential in MeV, including the rest mass.

property pair_rest_energy: float

Rest energy of the positrons and the electrons paired with them, erg cm^{-3} .

property entropy_per_nucleon: float

Entropy of the electrons and positrons per nucleon, in units of k_B .

property degeneracy_factor: float

How strongly the electrons screen compared with a classical gas.

$(d n / d \eta) / (n_- - n_+)$: one for a non-degenerate electron gas, falling towards zero as the electrons become degenerate, and above one when positron pairs add to the screening.

property screening_term: float

The electrons' share of the Debye sum, per nucleon.

Debye screening by the ions is $\sum Z^2 Y$; the electrons and positrons add $kT \, dn/d\mu / (\rho \, N_A)$, which is this value. It equals Y_e for non-degenerate electrons and tends to zero for degenerate ones.

`nucnetpy.electrons.electron_chemical_potential ( t9, rho, ye )`

Return η , the electron chemical potential over kT without rest mass.

It is the value for which the net electron density $n_- - n_+$ equals $\rho \, N_A \, Y_e$.

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)

Return type float

`nucnetpy.electrons.electron_gas ( t9, rho, ye )`

Return the full [ElectronGas](#) state at $(t9, \rho, ye)$.

$t9$ is the temperature in 10^9 K, ρ the density in g cm^{-3} and ye the net number of electrons per nucleon.

Parameters

- **t9** (*float*)
- **rho** (*float*)
- **ye** (*float*)

Return type [ElectronGas](#)

Validation and regression utilities for replacing NucNet Tools workflows.

class `nucnetpy.validation.ValidationIssue ( level: 'str', code: 'str', message: 'str' )`

Bases: `object`

Parameters

- **level** (*str*)
- **code** (*str*)
- **message** (*str*)

level: str

code: str

message: str

`nucnetpy.validation.validate_network ( network, strict=False )`

Parameters

- **network** ([Network](#))
- **strict** (*bool*)

Return type *List[ValidationIssue]*

`nucnetpy.validation.validate_zone ( zone, network=None, atol=1e-06 )`

Parameters

- **zone** (*Zone*)
- **network** (*Network* | *None*)
- **atol** (*float*)

Return type *List[ValidationIssue]*

`nucnetpy.validation.compare_abundances ( a, b, rtol=1e-06, atol=1e-30 )`

Parameters

- **a** (*Mapping[str, float]*)
- **b** (*Mapping[str, float]*)
- **rtol** (*float*)
- **atol** (*float*)

Return type *Dict[str, float]*

`nucnetpy.validation.max_relative_abundance_error ( a, b, atol=1e-30 )`

Parameters

- **a** (*Mapping[str, float]*)
- **b** (*Mapping[str, float]*)
- **atol** (*float*)

Return type *float*

`nucnetpy.validation.regression_summary ( reference, candidate )`

Parameters

- **reference** (*Mapping[str, float]*)
- **candidate** (*Mapping[str, float]*)

Return type *Dict[str, object]*

Pure-Python rate modification hooks replacing user/rate_modifiers.*.

class `nucnetpy.rate_modifiers.RateModifierRegistry ( modifiers: 'Dict[str, RateModifier]'` *=<factory>* `)`

Bases: *object*

Parameters **modifiers** (*Dict[str, Callable[[object, float, float, float] | None, Mapping[str, float]], float]*)

modifiers: *Dict[str, Callable[[object, float, float, float] | None, Mapping[str, float]], float]*

register (*key, func*)

Parameters

- **key** (*str*)
- **func** (*Callable[[object, float, float, float] | None, Mapping[str, float]], float]*)

Return type *None*

factor (*reaction, t9, rho, ye, abundances*)

Parameters

- **t9** (*float*)
- **rho** (*float*)

- **ye** (*float* | *None*)
- **abundances** (*Mapping*[*str*, *float*])

Return type *float*

`nucnetpy.rate_modifiers.constant_factor ( value )`

Parameters **value** (*float*)

Return type *Callable*[[*object*, *float*, *float*, *float* | *None*, *Mapping*[*str*, *float*]], *float*]

`nucnetpy.rate_modifiers.exp_temperature_factor ( alpha, power=1.0 )`

Parameters • **alpha** (*float*)

 • **power** (*float*)

Return type *Callable*[[*object*, *float*, *float*, *float* | *None*, *Mapping*[*str*, *float*]], *float*]

Graph export helpers replacing the Graphviz-oriented examples.

`nucnetpy.graph.reaction_network_dot ( network, min_rate=0.0, t9=1.0, rho=1.0 )`

Parameters • **network** (*Network*)

 • **min_rate** (*float*)

 • **t9** (*float*)

 • **rho** (*float*)

Return type *str*

Physical constants used by nucnetpy.

The values are SI/cgs mixed in the conventional astrophysical style used by network codes. They are intentionally centralized so advanced users can swap or audit constants when comparing with libnucnet/NucNet Tools builds.

Command line

Installing the package puts a `nucnetpy` command on the path. This page is generated from the argument parser by `docs/manual/generate_cli_reference.py`, so it matches the code.

```
usage: nucnetpy [-h]

{summary, print-output, largest-x, zone-abundances, zone-properties, element-abundances, rates, flows, y
...

Pure-Python NucNet Tools replacement commands

positional arguments:

{summary, print-output, largest-x, zone-abundances, zone-properties, element-abundances, rates, flows, y
net-flows              forward, detailed-balance reverse, and net fluxes
  charge-flows          per-reaction dYe/dt contributions
  timescales            shortest species timescales Y/|dY/dt|
  entropy-generation    dS/dt per nucleon in k_B/s
  qse                   constrained cluster equilibrium (libnuceq)
  jina-summary          summarise separate JINA nuclide and reaction XML files
  jina-combine          combine JINA nuclide and reaction XML files into one
                        nucnetpy XML file

options:
  -h, --help            show this help message and exit
```

7.1 summary

```
usage: nucnetpy summary [-h] xml

positional arguments:
  xml

options:
  -h, --help  show this help message and exit
```

7.2 print-output

```
usage: nucnetpy print-output [-h] [--max-zones MAX_ZONES]
                             [--min-abundance MIN_ABUNDANCE]
                             xml
```

```
positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --max-zones MAX_ZONES
  --min-abundance MIN_ABUNDANCE
```

7.3 largest-x

```
usage: nucnetpy largest-x [-h] [-n N] [--zone-index ZONE_INDEX]
                          [--min-x MIN_X]
                          xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  -n N
  --zone-index ZONE_INDEX
  --min-x MIN_X
```

7.4 zone-abundances

```
usage: nucnetpy zone-abundances [-h] [--zone-index ZONE_INDEX]
                                [--min-abundance MIN_ABUNDANCE]
                                xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --zone-index ZONE_INDEX
  --min-abundance MIN_ABUNDANCE
```

7.5 zone-properties

```
usage: nucnetpy zone-properties [-h] [--zone-index ZONE_INDEX] xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --zone-index ZONE_INDEX
```


7.6 element-abundances

```
usage: nucnetpy element-abundances [-h] [--zone-index ZONE_INDEX] xml element

positional arguments:
  xml
  element

options:
  -h, --help            show this help message and exit
  --zone-index ZONE_INDEX
```

7.7 rates

```
usage: nucnetpy rates [-h] [--t9 T9] [--rho RHO] [--min-rate MIN_RATE] xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --t9 T9
  --rho RHO
  --min-rate MIN_RATE
```

7.8 flows

```
usage: nucnetpy flows [-h] [--zone-index ZONE_INDEX] [--t9 T9] [--rho RHO]
                    [--min-flow MIN_FLOW] xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --zone-index ZONE_INDEX
  --t9 T9
  --rho RHO
  --min-flow MIN_FLOW
```

7.9 ydot

```
usage: nucnetpy ydot [-h] [--zone-index ZONE_INDEX] [--t9 T9] [--rho RHO]
                    [--min-abs MIN_ABS] xml

positional arguments:
  xml

options:
```

```
-h, --help            show this help message and exit
--zone-index ZONE_INDEX
--t9 T9
--rho RHO
--min-abs MIN_ABS
```

7.10 net-flows

```
usage: nucnetpy net-flows [-h] [--zone-index ZONE_INDEX] [--t9 T9] [--rho RHO]
                        [--min-flow MIN_FLOW]
                        xml
```

positional arguments:
xml

options:
-h, --help show this help message and exit
--zone-index ZONE_INDEX
--t9 T9
--rho RHO
--min-flow MIN_FLOW

7.11 charge-flows

```
usage: nucnetpy charge-flows [-h] [--zone-index ZONE_INDEX] [--t9 T9]
                             [--rho RHO] [--min-flow MIN_FLOW]
                             xml
```

positional arguments:
xml

options:
-h, --help show this help message and exit
--zone-index ZONE_INDEX
--t9 T9
--rho RHO
--min-flow MIN_FLOW

7.12 timescales

```
usage: nucnetpy timescales [-h] [--zone-index ZONE_INDEX] [--t9 T9]
                           [--rho RHO] [-n N]
                           xml
```

positional arguments:
xml

options:
-h, --help show this help message and exit
--zone-index ZONE_INDEX
--t9 T9
--rho RHO
-n N

7.13 entropy-generation

```
usage: nucnetpy entropy-generation [-h] [--zone-index ZONE_INDEX] [--t9 T9]
                                   [--rho RHO]
                                   xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --zone-index ZONE_INDEX
  --t9 T9
  --rho RHO
```

7.14 conservation

```
usage: nucnetpy conservation [-h] [--max MAX] xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --max MAX
```

7.15 remove-duplicates

```
usage: nucnetpy remove-duplicates [-h] xml output

positional arguments:
  xml
  output

options:
  -h, --help            show this help message and exit
```

7.16 remove-invalid

```
usage: nucnetpy remove-invalid [-h] xml output

positional arguments:
  xml
  output

options:
  -h, --help            show this help message and exit
```

7.17 export-zone-xml

```
usage: nucnetpy export-zone-xml [-h] [--zone-index ZONE_INDEX] xml output

positional arguments:
  xml
  output

options:
  -h, --help            show this help message and exit
  --zone-index ZONE_INDEX
```

7.18 reactions-latex

```
usage: nucnetpy reactions-latex [-h] xml output

positional arguments:
  xml
  output

options:
  -h, --help            show this help message and exit
```

7.19 species-history

```
usage: nucnetpy species-history [-h] xml species

positional arguments:
  xml
  species

options:
  -h, --help            show this help message and exit
```

7.20 evolve-zone

```
usage: nucnetpy evolve-zone [-h] [--zone-index ZONE_INDEX] [--t0 T0] [--t1 T1]
                             [--steps STEPS] [--t9 T9] [--rho RHO]
                             [--method
{bdf, radau, lsoda, rk45, dop853, rk4, euler, implicit_euler, backward_euler}]
                             [--log-time] [--min-abundance MIN_ABUNDANCE]
                             xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --zone-index ZONE_INDEX
  --t0 T0
  --t1 T1
```

```

--steps STEPS
--t9 T9
--rho RHO
--method {bdf, radau, lsoda, rk45, dop853, rk4, euler, implicit_euler, backward_euler}
--log-time
--min-abundance MIN_ABUNDANCE

```

7.21 energy-generation

```

usage: nucnetpy energy-generation [-h] [--zone-index ZONE_INDEX] [--t9 T9]
                                   [--rho RHO]
                                   xml

```

positional arguments:
xml

options:
-h, --help show this help message **and** exit
--zone-index ZONE_INDEX
--t9 T9
--rho RHO

7.22 net-dot

```

usage: nucnetpy net-dot [-h] [-o OUTPUT] [--t9 T9] [--rho RHO]
                        [--min-rate MIN_RATE]
                        xml

```

positional arguments:
xml

options:
-h, --help show this help message **and** exit
-o OUTPUT, --output OUTPUT
--t9 T9
--rho RHO
--min-rate MIN_RATE

7.23 validate

```

usage: nucnetpy validate [-h] [--strict] [--max MAX] [--max-zones MAX_ZONES]
                        xml

```

positional arguments:
xml

options:
-h, --help show this help message **and** exit
--strict
--max MAX
--max-zones MAX_ZONES

7.24 nse

```
usage: nucnetpy nse [-h] --t9 T9 --rho RHO --ye YE [--coulomb]
                  [--min-abundance MIN_ABUNDANCE] [--min-x MIN_X]
                  xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --t9 T9
  --rho RHO
  --ye YE
  --coulomb              apply Bravo & Garcia-Senz Coulomb corrections
  --min-abundance MIN_ABUNDANCE
  --min-x MIN_X
```

7.25 qse

```
usage: nucnetpy qse [-h] --t9 T9 --rho RHO --ye YE [--cluster SP1,SP2,...:Y]
                  [--coulomb] [--min-abundance MIN_ABUNDANCE]
                  [--min-x MIN_X]
                  xml

positional arguments:
  xml

options:
  -h, --help            show this help message and exit
  --t9 T9
  --rho RHO
  --ye YE
  --cluster SP1,SP2,...:Y
                        cluster species and constrained total abundance;
                        repeatable
  --coulomb
  --min-abundance MIN_ABUNDANCE
  --min-x MIN_X
```

7.26 jina-summary

```
usage: nucnetpy jina-summary [-h] [--zones-xml ZONES_XML] [--show-invalid]
                             [--max-invalid MAX_INVALID]
                             nuclides_xml reactions_xml

positional arguments:
  nuclides_xml
  reactions_xml

options:
  -h, --help            show this help message and exit
  --zones-xml ZONES_XML
  --show-invalid
```

```
--max-invalid MAX_INVALID
```

7.27 jina-combine

```
usage: nucnetpy jina-combine [-h] [--zones-xml ZONES_XML]
                             nuclides_xml reactions_xml output_xml
```

positional arguments:

```
  nuclides_xml
  reactions_xml
  output_xml
```

options:

```
  -h, --help            show this help message and exit
  --zones-xml ZONES_XML
```

Conversion map from original NucNet Tools concepts to nucnetpy

Original area	Python replacement
Libnucnet__Nuc / species handling	nucnetpy.species.Species, Network.species
Libnucnet__Reac / reactions	nucnetpy.reactions.Reaction, ReactionNetwork, RateFit, TabularRate
ReacLib seven-parameter rates	RateFit.rate(t9)
Reaction duplicate/invalid utilities	ReactionNetwork.remove_duplicates, filter_valid, CLI remove-duplicates, remove-invalid
Zone XML and abundance output	Zone, Network.zones, nucnetpy.io.xml
Analysis examples (print_output, largest X, flows, ydot)	nucnetpy.analysis and CLI commands
Graph utilities	nucnetpy.graph.reaction_network_dot, CLI net-dot
Screening user callbacks	nucnetpy.screening with weak/intermediate models and solver callback hook
Weak-rate utilities	nucnetpy.weak.WeakRateTable, read_weak_table, solver weak-rate hook
libnuceq/NSE workflows	nucnetpy.nse.solve_nse, equilibrium_ratio
Matrix/Jacobian solver utilities	nucnetpy.solver.jacobian, jacobian_sparsity, SciPy BDF/Radau/LSODA
Multi-zone evolution	evolve_network_zones
HDF5 helper workflow	nucnetpy.io.hdf5

8.1 What is implemented

- Flexible XML parsing for common legacy structures and compact Python-written XML.
- Partition functions and optional properties are preserved.
- ReacLib, tabular, and constant rates are supported.
- NSE solves are available for network-level equilibrium initialization.
- Screening and weak-rate hooks are now solver-compatible.
- A sparse numerical Jacobian and sparsity pattern are available for stiff integration.
- Validation/regression helpers are included.

8.2 What still needs project-specific validation

A full replacement can now be used and extended, but exact reproduction of a historical C++ run requires comparing against your original input/output files. The main tunable points are:

1. exact XML schema variants used by your input files,
2. original mass excess and partition-function data,
3. exact screening callback selected in the original C++ build,
4. weak-rate table source and interpolation convention,
5. solver tolerance, timestep, and rate-modifier settings.

Blog workflow coverage

Mapping of the science workflows from Bradley Meyer’s NucNet Tools blog (<https://sourceforge.net/u/mbradle/blog/>, 94 posts, 2012–2019) to nucnetpy features. Installation/infrastructure posts (Cygwin, Boost, compilers, AWS, SourceForge housekeeping — 24 posts) do not apply to a pure-Python package and are omitted.

9.1 Covered

Blog workflow	nucnetpy feature
Running a first network calculation; changing input conditions; dynamic H burning	<code>evolve_zone</code> , <code>constant_thermo</code> , CLI <code>evolve-zone</code>
Analyzing a network calculation; printing properties and mass fractions	<code>analysis</code> , CLI <code>print-output</code> , <code>largest-x</code> , <code>zone-*</code>
Understanding/updating/validating nuclear and reaction data	<code>io.xml</code> , <code>io.jina</code> , <code>validate_network</code> , CLI <code>validate</code>
Merging nuclear and reaction data	<code>combine_jina_xml</code> , CLI <code>jina-combine</code>
Calculating reaction rates	<code>RateFit</code> , <code>TabularRate</code> , CLI <code>rates</code>
Comparing forward and reverse reaction rates	<code>reverse_rate</code>, <code>reverse_reaction</code> (detailed balance)
Computing reaction flows (forward + reverse)	<code>net_flows</code>, CLI <code>net-flows</code> ; forward-only: <code>flows</code>
Constructing the rate equations	<code>solver.rhs</code> , <code>jacobian</code> , <code>jacobian_sparsity</code>
Finding valid reactions; conservation	<code>invalid_reactions</code> , <code>conserves_a_z</code> , CLI <code>conservation</code>
Modifying reaction rates during a calculation; user-defined rate modifiers	<code>rate_modifiers</code>
Specifying an abundance in a network calculation	<code>Zone.set_abundance</code>
Selecting an input network; limiting the reaction network	<code>select_species (new)</code> , <code>limit_network</code>
Nuclide XPath / XSLT node-set extraction	<code>select_species (Z/A/element selection)</code>
Sorting species in a network calculation	<code>Network.species_names (Z, A order)</code>
Calculating nuclear statistical equilibrium	<code>solve_nse</code> , CLI <code>nse</code>

Comparing network calculations to equilibrium	<code>solve_nse + regression_summary</code>
Studying the nuclear partition function	<code>Species.partition</code> , partition-aware NSE
Understanding abundance moments	<code>abundance_moment</code>
Computing the number of heavy nuclei	<code>heavy_nuclei_abundance</code> (new)
Computing system timescales	<code>system_timescales</code> (new), CLI <code>timescales</code>
Computing charge-changing flows	<code>charge_changing_flows</code> (new), CLI <code>charge-flows</code>
Computing the s-process neutron exposure	<code>neutron_exposure</code> (new)
Plotting abundances versus nucleon number	<code>abundances_vs_nucleon_number</code>
Plotting isotopic abundances versus time	<code>species_history</code> , <code>EvolutionResult.mass_fraction_history</code>
Computing separation energies	<code>separation_energy</code> (proton branch fixed)
Computing/understanding the entropy generation rate (incl. weak decay)	<code>entropy_generation_rate</code> (new), CLI <code>entropy-generation</code>
Computing thermodynamic quantities	<code>thermo</code> , <code>electrons</code> (electron-positron gas: chemical potential, pressure, energy, entropy)
Defining your own thermodynamic (trajectory) function	<code>ThermoFunction</code> callables, <code>hydro.Trajectory</code>
Using a thermodynamic trajectory file	<code>read_trajectory</code>
Running a network calculation with simple hydrodynamics	<code>exponential_expansion</code>
Including electron screening; supplying your own screening function	<code>screening</code> , solver <code>screening=hook</code>
Adding fission to an r-process calculation	<code>fission_reaction</code> (new)
Running an r-process calculation	<code>evolve_zone + JINA XML network</code>
(n, γ)-(γ ,n) equilibrium studies	<code>reverse_reaction</code> to build photodisintegration partners; <code>solve_nse</code>
Creating flow diagrams (colours, scalings, currents)	<code>graph.reaction_network_dot</code> , CLI <code>net-dot</code> (basic)
Creating webnucleo reaction XML from text with Python	<code>read_reaclib_text</code> , <code>io.text</code> , <code>write_xml</code>
Using zone properties; global strings	<code>Zone.properties</code> , <code>Network.metadata</code>
Selecting a zone as input for a new calculation	CLI <code>export-zone-xml</code>
Controlling the frequency of writing output	<code>times</code> grid passed to <code>evolve_zone</code>
Using wnutils to analyse output	<code>analysis module + io.hdf5</code>
Weak decays / entropy for weak decay	<code>weak tables + charge_changing_flows</code>
Updating mass excesses in a nuclide XML	<code>edit Species.mass_excess</code> , <code>re-write_xml</code>

Counting matrix elements; arrow matrix storage	jacobian_sparsity (SciPy sparse replaces arrow solver)
--	--

9.2 Ported from the C++ source (r647)

These items were verified against the actual NucNet Tools SVN trunk (nucnet-tools-code r647, downloaded from SourceForge) and ported from the named C++ files:

Blog work-flow	C++ origin	nucnetpy feature
Studying Coulomb corrections to NSE; including NSE corrections in a network calculation	user/nse_corr.cpp (Bravo & García-Senz 1999, Ogata & Ichimaru constants)	coulomb module: gamma_e, species_coulomb_chemical_potential / _energy / _entropy, coulomb_nse_correction; solve_nse(..., nse_correction=...) hook
Computing NSE corrections to the entropy	user/nse_corr.cpp (species_coulomb_entropy)	coulomb_entropy_per_nucleon
Comparing network calculations to equilibrium; (n, γ)-(γ ,n) equilibrium; constraining the equilibrium	libnuceq clusters via examples/analysis/compare_equil.cpp	solve_qse with QSECluster constraints (one multiplier per cluster on top of μ_p/μ_n)
Calculating cluster flows	examples/analysis/compute_Ycdot.cpp	cluster_ydot, cluster_abundance

Creating /ana- lyzing inte- grated currents diagrams	user/flow_utilities.cpp (update_flow_currents: current += (f-r).dt)	integrated_currents an EvolutionResult over
Computing the entropy gener- ation rate (per reac- tion, with reverse flows)	user/flow_utilities.cpp (compute_entropy_generation_rate)	entropy_generation_rate(use_reverse=True) $-\sum(f-r)\cdot\ln(f/r)$, non-negative, vanishes at NSE; reaction_entropy_changes

9.3 Not ported (by design or still open)

Blog workflow	Status
Inverting the entropy to find the density	entropy_per_nucleon (ions with masses, spins and partition functions; electrons and positrons from nucnetpy.electrons; photons; optional Coulomb term), density_for_entropy, t9_for_entropy. The old proxies density_from_entropy and temperature_from_entropy are deprecated
Constant-entropy evolu- tion with self-consistent entropy inverter	constant_entropy_thermo(entropy, density, network) — a thermo callable that sets T9 from the current composition at each step; the reactions' heat is not fed back
Electron screening in non-degenerate matter with SkyNetScreening	the electron and positron term of the Debye sum is computed from the electron gas (ElectronGas.screening_term); pair_term over-rides it
Electron/positron chemical-potential terms in entropy generation (libstatmech)	open — weak reactions carry tabulated rates instead
Fission <i>cycling</i> studies (automatic fragment distributions)	partial — fission_reaction builds channels; distributions are user input
Downloading webnu- cleo XML files	use the URLs in the blog post; nucnetpy reads them directly
XML inclusion (XIn- clude) / XSLT trans- forms	use lxml externally, then read_xml

Pure Python port status

The C++ source contains helper files under `nnt/` and `user/`. NucNetPy provides Python equivalents for the practical computational layers rather than a wrapper around a compiled backend.

Original area	Python module	Status
<code>nnt/math.*</code>	<code>nucnetpy.mathutils</code>	Ported: linear, bilinear, 2-D interpolation behaviour
<code>wrappers/containers</code>	<code>core.py</code> , <code>species.py</code>	Ported core containers
<code>network/reaction utilities</code>	<code>reactions.py</code> , <code>validation.py</code>	Ported main rate /flow/conservation APIs
<code>screen.*</code>	<code>screening.py</code>	Pure-Python weak and intermediate screening; exact formula variants still need golden validation
<code>weak_utilities.*</code> , <code>two_d_weak_rates.*</code>	<code>weak.py</code>	Ported 2-D weak-rate table interpolation and weak source terms
<code>thermo.*</code>	<code>thermo.py</code> , <code>electrons.py</code> , <code>nse.py</code>	Entropy of ions, electrons and positrons, and photons; density and temperature from entropy; NSE. The electron gas is checked against closed-form limits and thermodynamic identities, not against a libstat-mech build
<code>matrix_solver.*</code> , <code>ilu_solvers.*</code>	<code>matrix_solver.py</code> , <code>solver.py</code>	Ported to SciPy sparse solvers and stiff integrators
<code>hydro.*</code> , <code>hydro_helper.*</code>	<code>hydro.py</code>	Ported trajectory interpolation and exponential expansion

<code>network_limiter.*</code>	<code>network_limiter.py</code>	Ported species/reaction subnetwork limiting
<code>nuclear_decay_utilities.*</code>	<code>decay.py</code>	Ported half-life /decay record tools
<code>neutrino_rate_functions.*</code>	<code>neutrino.py</code>	Ported neutrino luminosity /cross-section helpers
<code>hdf5_routines.*</code>	<code>io/hdf5.py</code>	Ported compact HDF5 save/load
<code>graph.h</code>	<code>graph.py</code>	Ported DOT export

10.1 What remains for strict numerical identity

No C++ is used in this package. To make the Python code exactly match a chosen original C++ build, add golden-output tests generated once from the original tools, then tune the pure-Python formulas and tolerances until those tests pass.

The framework for this now exists in-repo. `tests/test_golden_identity.py` implements the full recommended order below against golden files in `tests/golden/`, and `validation/generate_golden.py` regenerates those files:

1. XML round-trip tests for your real networks — `test_xml_round_trip`, `test_round_trip_is_a_fixed_point` (identity tests, no golden data needed).
2. Rate-by-rate ReacLib comparisons on a T9 grid — `test_reaclib_rates_on_t9_grid` vs `rates_reaclib.json`.
3. Screening-factor comparisons for your chosen user/screen.cpp model — `test_screening_factors` vs `screening.json`.
4. Weak-rate table comparisons — `test_weak_rate_table_interpolation` vs `weak_rates.json`.
5. One-zone RHS ydot comparisons at fixed state — `test_ydot_at_fixed_states` vs `ydot.json` (also asserts nucleon-number conservation of the RHS).
6. Full trajectory comparisons with identical time grids and tolerances — `test_full_trajectory` vs `trajectory.json` (deterministic fixed-step rk4), plus a solver cross-check that adaptive BDF agrees with the rk4 path.

Out of the box the goldens are self-consistent snapshots of the current Python numerics, so any future change to a formula fails the suite immediately. To validate against a specific C++ NucNet Tools build:

1. Replace the fixture inputs (`tests/golden/golden_network.xml`, `golden_weak_rates.txt`) with your production data, or keep them and reproduce the same inputs on the C++ side.
2. Run the same grids through the original tools and write their outputs into the data blocks of the JSON files (set `source` to your build id).
3. Loosen each file's `rtol/atol` to the agreement you require — the tolerances are read from the golden files, not hard-coded in the tests.

Exact numerical identity policy

NucNetPy is pure Python. It contains no C++ code, does not call the original NucNet Tools programs, and has no “exact backend”. It therefore cannot promise bit-for-bit agreement with a particular C++ build on its own.

11.1 Why identity cannot simply be assumed

Two codes that implement the same equations can still disagree in the last digits, or by more, because results depend on:

- the nuclear data: mass excesses, partition-function tables and their interpolation, and the rate fits;
- modelling choices: the screening prescription, how reverse rates are formed, and which species enter an equilibrium solve;
- numerical choices: the integrator, its tolerances and time grid, the order in which reactions are summed, and the linear-algebra library;
- the platform: compiler, optimisation flags and floating-point rounding.

Agreement has to be shown by comparison, not asserted.

11.2 How to compare with a C++ build you run yourself

The repository carries the machinery for this; the port status notes (`docs/PURE_PYTHON_PORT_STATUS.md`) describe it in detail.

1. Run the original NucNet Tools on the inputs in `tests/golden/` (or replace those inputs with your own network and run both codes on them).
2. Write the C++ outputs into the data blocks of the JSON files in `tests/golden/`, and set each file’s `source` to a label for your build.
3. Set each file’s `rtol` and `atol` to the agreement you need. The tests read the tolerances from the files, so no test code has to change.
4. Run `pytest tests/test_golden_identity.py`. A failure names the quantity, the value from each code and the tolerance.

Out of the box those files hold snapshots of NucNetPy’s own output, so the same tests also catch any unintended change to the Python numerics. `validation/generate_golden.py` regenerates the snapshots after an intended change.

11.3 Development rule

When a C++ example is converted to Python, add a golden-output test for it:

1. Run the original C++ program on a fixed input file and save its output.
2. Run the Python equivalent on the same input.
3. Compare with zero tolerance first. If the platforms differ in the last bits, record the tolerance you need and why.

Only modules that pass such tests should be described as exact replacements.

JINA XML workflow in nucnetpy

The original C++ NucNet Tools examples usually begin from JINA/libnucnet XML files. The Python port follows the same idea, but uses a Python object model.

12.1 Files

Use your stored XML files as follows:

- `jina_nuclides.xml`: nuclides/species, including Z , A , mass excess, spin, and partition data.
- `jina_reactions.xml`: reactions and ReacLib rate coefficients.
- optional `zones.xml`: initial abundances or output zones.

12.2 Python usage

```
from nucnetpy import read_jina_xml

net = read_jina_xml("jina_nuclides.xml", "jina_reactions.xml")
print(net.validate())
```

With zones:

```
net = read_jina_xml(
    "jina_nuclides.xml",
    "jina_reactions.xml",
    zones_xml="initial_zones.xml",
)
```

12.3 Command-line usage

```
nucnetpy jina-summary jina_nuclides.xml jina_reactions.xml
nucnetpy jina-combine jina_nuclides.xml jina_reactions.xml combined_network.xml
nucnetpy rates combined_network.xml --t9 1.0 --rho 1e5
nucnetpy conservation combined_network.xml
```

12.4 Important numerical point

To approach numerical identity with the original C++ tools, use the exact same JINA XML files, do

not reorder or filter reactions unless intentionally testing that operation, and compare rates/flows at fixed `T9`, `rho`, and abundance values.

Where to start

If you have never used the package, read [Installation](#) and [Concepts](#), then work through the tutorial notebooks in `notebooks/`, which are the fastest way in.

If you are migrating an existing NucNet Tools workflow, [Conversion map from original NucNet Tools concepts to nucnetpy](#) maps the original concepts onto this API and [Blog workflow coverage](#) maps each blog workflow onto the feature that replaces it.

If you are about to run a production calculation, read [Pitfalls](#) first. It describes two behaviours that produce plausible but wrong answers without raising an error.

Citing

Please cite the archived release ([10.5281/zenodo.20756798](https://zenodo.org/record/20756798)) and acknowledge the NucNet Tools and libnucnet ecosystem, whose data model and workflows this package reimplements, together with the JINA reaction-rate database that supplies the nuclear data.

n

nucnetpy

- nucnetpy.analysis, 41
- nucnetpy.constants, 56
- nucnetpy.core, 16
- nucnetpy.coulomb, 31
- nucnetpy.decay, 39
- nucnetpy.detailed_balance, 32
- nucnetpy.electrons, 52
- nucnetpy.graph, 56
- nucnetpy.hydro, 47
- nucnetpy.io.jina, 46
- nucnetpy.io.xml, 45
- nucnetpy.network_limiter, 47
- nucnetpy.neutrino, 40
- nucnetpy.nse, 27
- nucnetpy.qse, 29
- nucnetpy.rate_modifiers, 55
- nucnetpy.reactions, 18
- nucnetpy.screening, 35
- nucnetpy.solver, 23
- nucnetpy.species, 15
- nucnetpy.thermo, 48
- nucnetpy.validation, 54
- nucnetpy.weak, 37

A

abundance_from_mass_fraction() (in module nucnetpy.species), 16
 abundance_matrix() (nucnetpy.core.Network method), 18
 abundance_moment() (in module nucnetpy.-analysis), 41
 abundances (nucnetpy.core.Zone attribute), 17
 abundances (nucnetpy.nse.NSEResult attribute), 28
 abundances (nucnetpy.qse.QSEResult attribute), 30
 abundances (nucnetpy.screening.ScreeningContext attribute), 35
 abundances_for_element() (in module nucnetpy.analysis), 41
 abundances_vs_nucleon_number() (in module nucnetpy.analysis), 41
 add() (nucnetpy.reactions.ReactionNetwork method), 22
 add_decay_records() (in module nucnetpy.decay), 40
 add_species() (nucnetpy.core.Network method), 18
 add_zone() (nucnetpy.core.Network method), 18
 analytic_jacobian() (in module nucnetpy.-solver), 25

B

bare_rate() (nucnetpy.reactions.Reaction method), 21
 beta (nucnetpy.electrons.ElectronGas attribute), 53
 branch (nucnetpy.decay.DecayRecord attribute), 39

C

chapter (nucnetpy.reactions.RateFit attribute), 19

charge_changing_flows() (in module nucnetpy.-analysis), 43
 chemical_potential() (nucnetpy.screening.SkyNetScreening method), 37
 cluster_abundance() (in module nucnetpy.qse), 30
 cluster_ydot() (in module nucnetpy.qse), 30
 code (nucnetpy.validation.ValidationIssue attribute), 54
 coefficients (nucnetpy.reactions.RateFit attribute), 19
 combine_jina_xml() (in module nucnetpy.io.jina), 47
 compare_abundances() (in module nucnetpy.-validation), 55
 compare_rates() (in module nucnetpy.analysis), 42
 compute_yedot() (in module nucnetpy.weak), 39
 computed_je (nucnetpy.nse.NSEResult property), 28
 computed_je (nucnetpy.qse.QSEResult property), 30
 conserves_a_z() (nucnetpy.reactions.Reaction method), 21
 consistent_reverse_network() (in module nucnetpy.detailed_balance), 33
 constant_entropy_thermo() (in module nucnetpy.thermo), 51
 constant_factor() (in module nucnetpy.rate_-modifiers), 56
 constant_rate (nucnetpy.reactions.Reaction attribute), 20
 constant_thermo() (in module nucnetpy.solver), 24
 constraint (nucnetpy.qse.QSECluster attribute), 29
 coulomb (nucnetpy.thermo.EntropyPerNucleon attribute), 49
 coulomb_entropy_per_nucleon() (in module nucnetpy.coulomb), 32
 count (nucnetpy.reactions.ReactionParticipant attribute), 19

`cross_section()` (nucnetpy.neutrino.Neutrino-Quantity method), 40

D

`daughter` (nucnetpy.decay.DecayRecord attribute), 39

`daughter` (nucnetpy.weak.WeakRateTable attribute), 38

`debye_radius()` (in module nucnetpy.screening), 35

`decay_constant_from_half_life()` (in module nucnetpy.decay), 40

`DecayRecord` (class in nucnetpy.decay), 39

`degeneracy_factor` (nucnetpy.electrons.ElectronGas property), 54

`density()` (nucnetpy.core.Zone method), 17

`density_for_entropy()` (in module nucnetpy.thermo), 50

`density_from_entropy()` (in module nucnetpy.thermo), 52

`DetailedBalanceOptions` (class in nucnetpy.reactions), 23

`dn_data` (nucnetpy.electrons.ElectronGas attribute), 53

E

`electron_chemical_potential()` (in module nucnetpy.electrons), 54

`electron_gas()` (in module nucnetpy.electrons), 54

`electron_number_density()` (in module nucnetpy.screening), 35

`electron_term()` (nucnetpy.screening.SkyNetScreening method), 37

`ElectronGas` (class in nucnetpy.electrons), 53

`electrons` (nucnetpy.thermo.EntropyPerNucleon attribute), 49

`element` (nucnetpy.species.Species property), 16

`element_abundances()` (in module nucnetpy.analysis), 41

`energy` (nucnetpy.electrons.ElectronGas attribute), 53

`energy_generation_rate()` (in module nucnetpy.analysis), 42

`ensure_species()` (nucnetpy.core.Network method), 18

`entropy_density` (nucnetpy.electrons.ElectronGas attribute), 53

`entropy_generation_rate()` (in module nucnetpy.analysis), 44

`entropy_ideal_ions()` (in module nucnetpy.thermo), 51

`entropy_per_nucleon` (nucnetpy.electrons.ElectronGas property), 54

`entropy_per_nucleon()` (in module nucnetpy.thermo), 50

`EntropyPerNucleon` (class in nucnetpy.thermo),

49

`equilibrium_ratio` (nucnetpy.reactions.DetailedBalanceOptions attribute), 23

`equilibrium_ratio()` (in module nucnetpy.nse), 28

`eta` (nucnetpy.electrons.ElectronGas attribute), 53

`EvolutionResult` (class in nucnetpy.solver), 24

`evolve_network_zones()` (in module nucnetpy.solver), 26

`evolve_zone()` (in module nucnetpy.solver), 26

`exp_temperature_factor()` (in module nucnetpy.rate_modifiers), 56

`exponential_expansion()` (in module nucnetpy.hydro), 48

`extend()` (nucnetpy.reactions.ReactionNetwork method), 22

F

`factor()` (nucnetpy.rate_modifiers.RateModifierRegistry method), 55

`factor()` (nucnetpy.screening.SkyNetScreening method), 37

`filter_valid()` (nucnetpy.reactions.ReactionNetwork method), 22

`final_abundances` (nucnetpy.solver.EvolutionResult property), 24

`fission_reaction()` (in module nucnetpy.decay), 40

`flows()` (in module nucnetpy.analysis), 42

`flows()` (nucnetpy.reactions.ReactionNetwork method), 22

`flux()` (nucnetpy.reactions.Reaction method), 21

`from_columns()` (nucnetpy.hydro.Trajectory class method), 48

`from_names()` (nucnetpy.reactions.Reaction class method), 20

G

`gamma_e()` (in module nucnetpy.coulomb), 31

`geometric_flux_rate()` (in module nucnetpy.neutrino), 41

`get_abundance()` (nucnetpy.core.Zone method), 17

`graboske_intermediate_factor()` (in module nucnetpy.screening), 36

H

`half_life` (nucnetpy.decay.DecayRecord attribute), 39

`heavy_nuclei_abundance()` (in module nucnetpy.analysis), 43

I

`ideal_gas_pressure()` (in module nucnetpy.thermo), 49

integrated_currents() (in module nucnetpy.-analysis), 44
 invalid_reactions() (nucnetpy.reactions.ReactionNetwork method), 22
 ion_entropy_per_nucleon() (in module nucnetpy.thermo), 49
 ion_strength (nucnetpy.screening.ScreeningContext property), 36
 ion_strength() (in module nucnetpy.screening), 35
 ions (nucnetpy.thermo.EntropyPerNucleon attribute), 49
 is_massless() (in module nucnetpy.species), 15

J

jacobian() (in module nucnetpy.solver), 25
 jacobian_sparsity() (in module nucnetpy.-solver), 26
 jina_database_summary() (in module nucnetpy.io.jina), 47

K

key (nucnetpy.reactions.Reaction property), 20

L

l0 (nucnetpy.neutrino.NeutrinoLuminosity attribute), 40
 label (nucnetpy.core.Zone attribute), 17
 label (nucnetpy.qse.QSECluster attribute), 29
 label (nucnetpy.reactions.RateFit attribute), 19
 label (nucnetpy.reactions.Reaction attribute), 20
 label (nucnetpy.weak.WeakRateTable attribute), 38
 lambdas (nucnetpy.qse.QSEResult attribute), 30
 largest_mass_fractions() (in module nucnetpy.-analysis), 41
 latex (nucnetpy.reactions.Reaction property), 21
 latex (nucnetpy.species.Species property), 16
 level (nucnetpy.validation.ValidationIssue attribute), 54
 limit_network() (in module nucnetpy.network_limiter), 47
 log10_xsec (nucnetpy.neutrino.NeutrinoQuantity attribute), 40
 log_equilibrium_constant() (in module nucnetpy.detailed_balance), 33
 log_interp (nucnetpy.reactions.TabularRate attribute), 19
 luminosity() (nucnetpy.neutrino.NeutrinoLuminosity method), 40

M

mass_excess (nucnetpy.species.Species attribute), 16
 mass_fraction() (in module nucnetpy.species), 16

mass_fraction_history() (nucnetpy.solver.EvolutionResult method), 25
 mass_fractions() (nucnetpy.core.Zone method), 17
 MASSLESS_SPECIES (in module nucnetpy.species), 15
 max_relative_abundance_error() (in module nucnetpy.validation), 55
 message (nucnetpy.nse.NSEResult attribute), 28
 message (nucnetpy.qse.QSEResult attribute), 30
 message (nucnetpy.solver.EvolutionResult attribute), 24
 message (nucnetpy.validation.ValidationIssue attribute), 54
 metadata (nucnetpy.core.Network attribute), 18
 metadata (nucnetpy.reactions.Reaction attribute), 20
 METHODS (in module nucnetpy.solver), 24
 mode (nucnetpy.decay.DecayRecord attribute), 39
 modifiers (nucnetpy.rate_modifiers.RateModifierRegistry attribute), 55

module

nucnetpy.analysis, 41
 nucnetpy.constants, 56
 nucnetpy.core, 16
 nucnetpy.coulomb, 31
 nucnetpy.decay, 39
 nucnetpy.detailed_balance, 32
 nucnetpy.electrons, 52
 nucnetpy.graph, 56
 nucnetpy.hydro, 47
 nucnetpy.io.jina, 46
 nucnetpy.io.xml, 45
 nucnetpy.network_limiter, 47
 nucnetpy.neutrino, 40
 nucnetpy.nse, 27
 nucnetpy.qse, 29
 nucnetpy.rate_modifiers, 55
 nucnetpy.reactions, 18
 nucnetpy.screening, 35
 nucnetpy.solver, 23
 nucnetpy.species, 15
 nucnetpy.thermo, 48
 nucnetpy.validation, 54
 nucnetpy.weak, 37
 mu_mev (nucnetpy.electrons.ElectronGas property), 53
 mu_n (nucnetpy.nse.NSEResult attribute), 28
 mu_n (nucnetpy.qse.QSEResult attribute), 29
 mu_p (nucnetpy.nse.NSEResult attribute), 28
 mu_p (nucnetpy.qse.QSEResult attribute), 29

N

n (nucnetpy.species.Species property), 16
 n_electrons (nucnetpy.electrons.ElectronGas attribute), 53

n_positrons (nucnetpy.electrons.ElectronGas attribute), 53
name (nucnetpy.species.Species attribute), 16
net_flows() (in module nucnetpy.detailed_balance), 34
Network (class in nucnetpy.core), 17
neutrino_loss() (nucnetpy.weak.WeakRateTable method), 38
NeutrinoLuminosity (class in nucnetpy.neutrino), 40
NeutrinoQuantity (class in nucnetpy.neutrino), 40
neutron_exposure() (in module nucnetpy.analysis), 43
nfev (nucnetpy.solver.EvolutionResult attribute), 24
njev (nucnetpy.solver.EvolutionResult attribute), 24
normalize_species_name() (in module nucnetpy.species), 16
nse_correction() (in module nucnetpy.-coulomb), 32
nse_prefactor() (in module nucnetpy.nse), 27
NSEResult (class in nucnetpy.nse), 27
nu_loss (nucnetpy.weak.WeakRateTable attribute), 38
nuclear_energy_generation_rate() (in module nucnetpy.analysis), 45
nuclear_energy_release() (in module nucnetpy.-analysis), 45
nuclear_reactants (nucnetpy.reactions.Reaction property), 21
nucnetpy.analysis
 module, 41
nucnetpy.constants
 module, 56
nucnetpy.core
 module, 16
nucnetpy.coulomb
 module, 31
nucnetpy.decay
 module, 39
nucnetpy.detailed_balance
 module, 32
nucnetpy.electrons
 module, 52
nucnetpy.graph
 module, 56
nucnetpy.hydro
 module, 47
nucnetpy.io.jina
 module, 46
nucnetpy.io.xml
 module, 45
nucnetpy.network_limiter
 module, 47
nucnetpy.neutrino
 module, 40

nucnetpy.nse
 module, 27
nucnetpy.qse
 module, 29
nucnetpy.rate_modifiers
 module, 55
nucnetpy.reactions
 module, 18
nucnetpy.screening
 module, 35
nucnetpy.solver
 module, 23
nucnetpy.species
 module, 15
nucnetpy.thermo
 module, 48
nucnetpy.validation
 module, 54
nucnetpy.weak
 module, 37

O

optional_properties (nucnetpy.core.Zone attribute), 17

P

pair_rest_energy (nucnetpy.electrons.ElectronGas property), 53
parent (nucnetpy.decay.DecayRecord attribute), 39
parent (nucnetpy.weak.WeakRateTable attribute), 38
parse() (nucnetpy.species.Species class method), 16
partition (nucnetpy.species.Species attribute), 16
photon_entropy_per_nucleon() (in module nucnetpy.thermo), 49
photons (nucnetpy.thermo.EntropyPerNucleon attribute), 49
pressure (nucnetpy.electrons.ElectronGas attribute), 53
products (nucnetpy.reactions.Reaction attribute), 20
properties (nucnetpy.core.Zone attribute), 17

Q

q_energy_erg_per_g() (in module nucnetpy.thermo), 52
q_value (nucnetpy.reactions.Reaction attribute), 20
QSECluster (class in nucnetpy.qse), 29
QSEResult (class in nucnetpy.qse), 29

R

radiation_pressure() (in module nucnetpy.ther-

mo), 49
 rate (nucnetpy.decay.DecayRecord property), 39
 rate() (nucnetpy.reactions.RateFit method), 19
 rate() (nucnetpy.reactions.Reaction method), 21
 rate() (nucnetpy.reactions.TabularRate method), 19
 rate() (nucnetpy.weak.WeakRateTable method), 38
 rate_fits (nucnetpy.reactions.Reaction attribute), 20
 rate_function (nucnetpy.reactions.Reaction attribute), 20
 RateFit (class in nucnetpy.reactions), 19
 RateModifierRegistry (class in nucnetpy.rate_modifiers), 55
 rates (nucnetpy.reactions.TabularRate attribute), 19
 rates (nucnetpy.weak.WeakRateTable attribute), 38
 rates() (nucnetpy.reactions.ReactionNetwork method), 22
 reactant_order (nucnetpy.reactions.Reaction property), 21
 reactants (nucnetpy.reactions.Reaction attribute), 20
 Reaction (class in nucnetpy.reactions), 20
 reaction() (nucnetpy.decay.DecayRecord method), 39
 reaction_entropy_changes() (in module nucnetpy.analysis), 43
 reaction_network_dot() (in module nucnetpy.graph), 56
 reaction_screening_factor() (in module nucnetpy.screening), 36
 ReactionNetwork (class in nucnetpy.reactions), 22
 ReactionParticipant (class in nucnetpy.reactions), 19
 reactions (nucnetpy.core.Network attribute), 18
 reactions (nucnetpy.reactions.ReactionNetwork attribute), 22
 read_jina_xml() (in module nucnetpy.io.jina), 46
 read_network_xml() (in module nucnetpy.io.xml), 46
 read_reaclib_text() (in module nucnetpy.reactions), 23
 read_trajectory() (in module nucnetpy.hydro), 48
 read_weak_table() (in module nucnetpy.weak), 38
 read_weak_tables() (in module nucnetpy.weak), 39
 read_xml() (in module nucnetpy.io.xml), 46
 read_xml_root() (in module nucnetpy.io.xml), 46
 read_xml_string() (in module nucnetpy.io.xml), 46
 register() (nucnetpy.rate_modifiers.RateModi-

fierRegistry method), 55
 regression_summary() (in module nucnetpy.validation), 55
 remove_duplicates() (nucnetpy.reactions.ReactionNetwork method), 22
 reverse_flux() (in module nucnetpy.detailed_balance), 34
 reverse_rate() (in module nucnetpy.detailed_balance), 33
 reverse_rate_from_detailed_balance() (in module nucnetpy.reactions), 23
 reverse_reaction() (in module nucnetpy.detailed_balance), 33
 rho (nucnetpy.electrons.ElectronGas attribute), 53
 rho (nucnetpy.hydro.Trajectory attribute), 48
 rho (nucnetpy.nse.NSEResult attribute), 27
 rho (nucnetpy.qse.QSEResult attribute), 29
 rho (nucnetpy.reactions.DetailedBalanceOptions attribute), 23
 rho (nucnetpy.screening.ScreeningContext attribute), 35
 rho_ye (nucnetpy.weak.WeakRateTable attribute), 38
 rhs() (in module nucnetpy.solver), 25

S

screening_term (nucnetpy.electrons.ElectronGas property), 54
 ScreeningContext (class in nucnetpy.screening), 35
 select_species() (in module nucnetpy.network_limiter), 47
 separation_energy() (in module nucnetpy.analysis), 42
 set_abundance() (nucnetpy.core.Zone method), 17
 SkyNetScreening (class in nucnetpy.screening), 36
 solve_nse() (in module nucnetpy.nse), 28
 solve_qse() (in module nucnetpy.qse), 30
 sound_speed_ideal_gamma() (in module nucnetpy.thermo), 52
 source (nucnetpy.reactions.Reaction attribute), 20
 source (nucnetpy.species.Species attribute), 16
 Species (class in nucnetpy.species), 15
 species (nucnetpy.core.Network attribute), 18
 species (nucnetpy.qse.QSECluster attribute), 29
 species (nucnetpy.reactions.ReactionParticipant attribute), 19
 species (nucnetpy.solver.EvolutionResult attribute), 24
 species_coulomb_chemical_potential() (in module nucnetpy.coulomb), 31
 species_coulomb_energy() (in module nucnetpy.coulomb), 31

species_coulomb_entropy() (in module nucnetpy.coulomb), 32
species_from_za() (in module nucnetpy.species), 16
species_history() (in module nucnetpy.analysis), 42
species_map (nucnetpy.screening.ScreeningContext attribute), 35
species_names() (nucnetpy.core.Network method), 18
species_names() (nucnetpy.reactions.ReactionNetwork method), 23
species_without_nuclear_data() (nucnetpy.core.Network method), 18
spin (nucnetpy.species.Species attribute), 16
statistical_factor() (nucnetpy.reactions.Reaction method), 21
stoichiometry() (nucnetpy.reactions.Reaction method), 21
string (nucnetpy.reactions.Reaction property), 21
success (nucnetpy.nse.NSEResult attribute), 28
success (nucnetpy.qse.QSEResult attribute), 30
success (nucnetpy.solver.EvolutionResult attribute), 24
system_timescales() (in module nucnetpy.analysis), 43

T

t9 (nucnetpy.electrons.ElectronGas attribute), 53
t9 (nucnetpy.hydro.Trajectory attribute), 48
t9 (nucnetpy.nse.NSEResult attribute), 27
t9 (nucnetpy.qse.QSEResult attribute), 29
t9 (nucnetpy.reactions.DetailedBalanceOptions attribute), 23
t9 (nucnetpy.reactions.TabularRate attribute), 19
t9 (nucnetpy.screening.ScreeningContext attribute), 35
t9 (nucnetpy.weak.WeakRateTable attribute), 38
t9_for_entropy() (in module nucnetpy.thermo), 51
tabular_rate (nucnetpy.reactions.Reaction attribute), 20
TabularRate (class in nucnetpy.reactions), 19
tau (nucnetpy.neutrino.NeutrinoLuminosity attribute), 40
temperature (nucnetpy.neutrino.NeutrinoQuantity attribute), 40
temperature9() (nucnetpy.core.Zone method), 17
temperature_from_entropy() (in module nucnetpy.thermo), 52
thermo() (nucnetpy.hydro.Trajectory method), 48
time (nucnetpy.hydro.Trajectory attribute), 48

time (nucnetpy.solver.EvolutionResult attribute), 24
time_grid() (in module nucnetpy.solver), 24
total (nucnetpy.thermo.EntropyPerNucleon property), 49
Trajectory (class in nucnetpy.hydro), 48

U

update() (nucnetpy.screening.SkyNetScreening method), 37

V

validate() (nucnetpy.core.Network method), 18
validate_network() (in module nucnetpy.validation), 54
validate_zone() (in module nucnetpy.validation), 55
ValidationIssue (class in nucnetpy.validation), 54

W

weak_screening_factor() (in module nucnetpy.screening), 36
weak_source_terms() (in module nucnetpy.weak), 39
WeakRateTable (class in nucnetpy.weak), 38
write_xml() (in module nucnetpy.io.xml), 46
write_zone_xml() (in module nucnetpy.io.xml), 46

X

xsum (nucnetpy.nse.NSEResult property), 28
xsum (nucnetpy.qse.QSEResult property), 30
xsum() (nucnetpy.core.Zone method), 17

Y

y (nucnetpy.solver.EvolutionResult attribute), 24
ydot() (in module nucnetpy.analysis), 42
ydot() (nucnetpy.reactions.ReactionNetwork method), 22
ye (nucnetpy.electrons.ElectronGas attribute), 53
ye (nucnetpy.nse.NSEResult attribute), 27
ye (nucnetpy.qse.QSEResult attribute), 29
ye (nucnetpy.reactions.DetailedBalanceOptions attribute), 23
ye (nucnetpy.screening.ScreeningContext attribute), 35
ye() (nucnetpy.core.Zone method), 17
ysum() (nucnetpy.core.Zone method), 17

Z

z (nucnetpy.species.Species attribute), 16
Zone (class in nucnetpy.core), 16
zone() (nucnetpy.core.Network method), 18

zone() (nucnetpy.nse.NSEResult method), [28](#)
zone() (nucnetpy.qse.QSEResult method), [30](#)
zone() (nucnetpy.solver.EvolutionResult
method), [25](#)
zone_thermo() (in module nucnetpy.solver), [24](#)
zones (nucnetpy.core.Network attribute), [18](#)

